



In this course, we will build professional habits for using Codex efficiently while maintaining control over code quality. The lessons focus on practical workflows that make AI-assisted development more reliable, repeatable, and scalable.

What we will learn

We will begin by configuring automated verification for Codex output. These checks help ensure that AI-generated code meets the quality standards of the project before we consider a task complete.

We will also develop a systematic debugging workflow. This includes pasting errors directly into Codex, evaluating the proposed fix, and saving successful solutions as reusable skills when they can help with future tasks.

Review remains an essential part of AI-assisted development. We will learn to inspect AI-generated changes and diffs with the same rigor that we would apply to human-written code, helping us protect the integrity of the project.

As our tasks become more complex, we will use subagents and parallel sessions to divide work, manage context, and scale our development workflow. We will also create hooks, which are user-defined handlers that run automatically during specific lifecycle events outside the agent loop. This allows quality checks and other safeguards to run without relying on us to request them each time.

Throughout the course, we will establish habits that keep Codex sessions efficient, controlled, and consistently productive.

Recommended knowledge

Before continuing, we should already have hands-on experience with Codex. This includes installing Codex, starting sessions, writing prompts, and reviewing or reverting changes with Git. If these tasks are unfamiliar, it is worth practicing them before beginning the course.

We should also be comfortable working in a terminal and using basic Git commands, including git diff, git status, git commit, and git restore. These operations will appear frequently throughout the course.

Project and technical setup

The course includes starter projects, but we will gain the most value if we also have an active project where we can apply each technique immediately.

Our development environment should include Node.js version 18 or later and a correctly configured Codex installation. We will also need either an OpenAI API key or a ChatGPT Pro or Max subscription.

Using the Dev Shop Starter Project

Before the next lesson, make sure the Dev Shop starter project is on your computer and running locally.

- **Use the starter project files for the lessons.** The complete project files are there as a reference if you want to compare your work later.

Step 1: Download the project files

Open the **Course Files** tab for this course and download:

- **Starter Projects - Codex Workflows** Use this to follow the course exercises.
- **Complete Projects - Codex Workflows** Use this only as a reference.

After downloading the starter ZIP, unzip it on your computer.

Step 2: Open the Shopping Cart project

For the next lesson, open this folder in Visual Studio Code:

```
? Starter Projects - Codex Workflows
  ? Dev Shop
    ? react_project ? open this folder
```

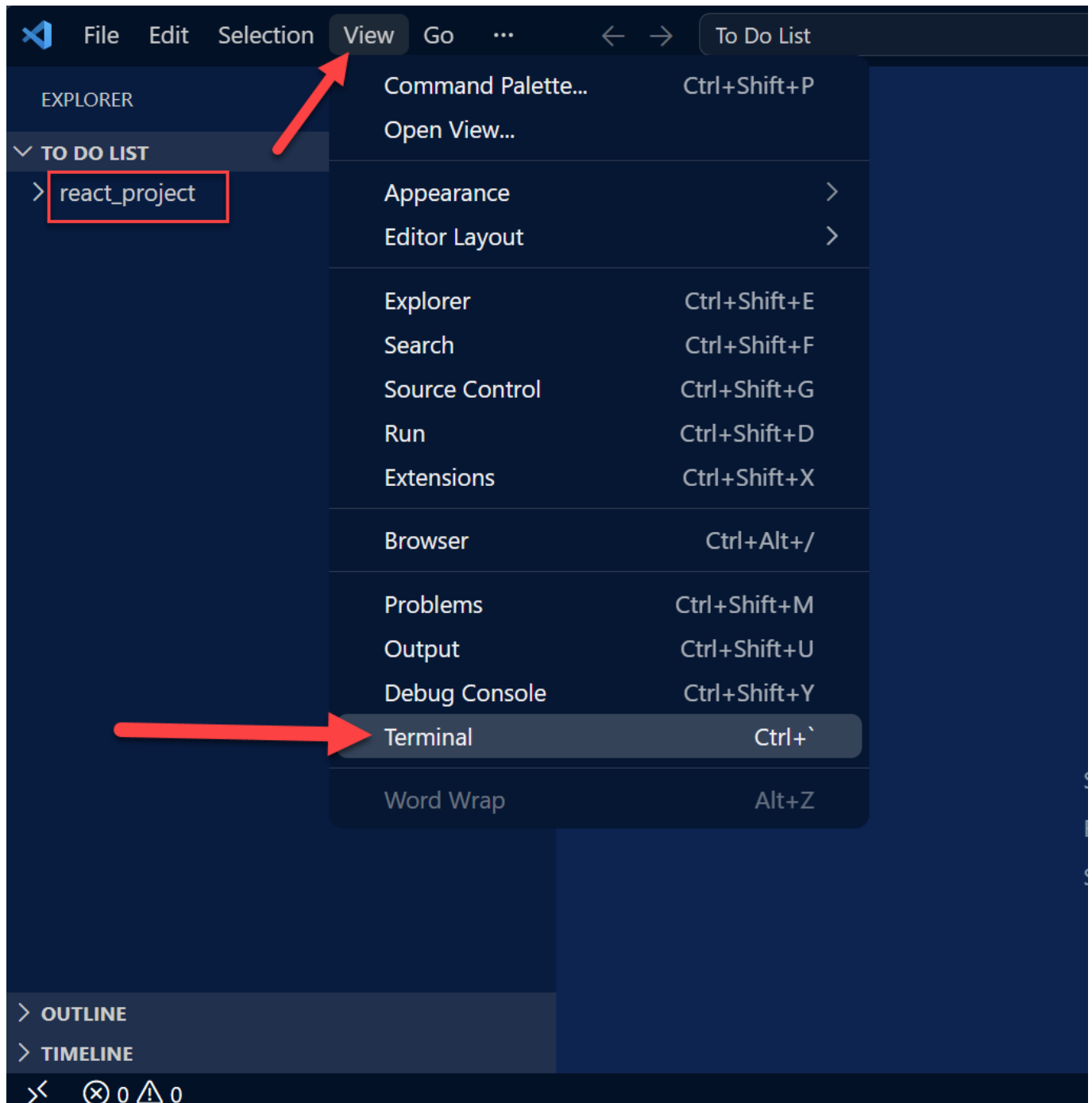
Make sure you open the react_project folder itself. This is the folder that contains package.json.

- **What is package.json?** It is a small settings file for the project. It tells Node.js which tools the app needs and which commands you can run, such as npm run dev.

Step 3: Run the app locally

- **Node.js note.** Node.js is needed to run these React project files locally.

Open the VS Code terminal while you are inside the react_project folder:



Then run npm like so:

```
npm install  
npm run dev
```

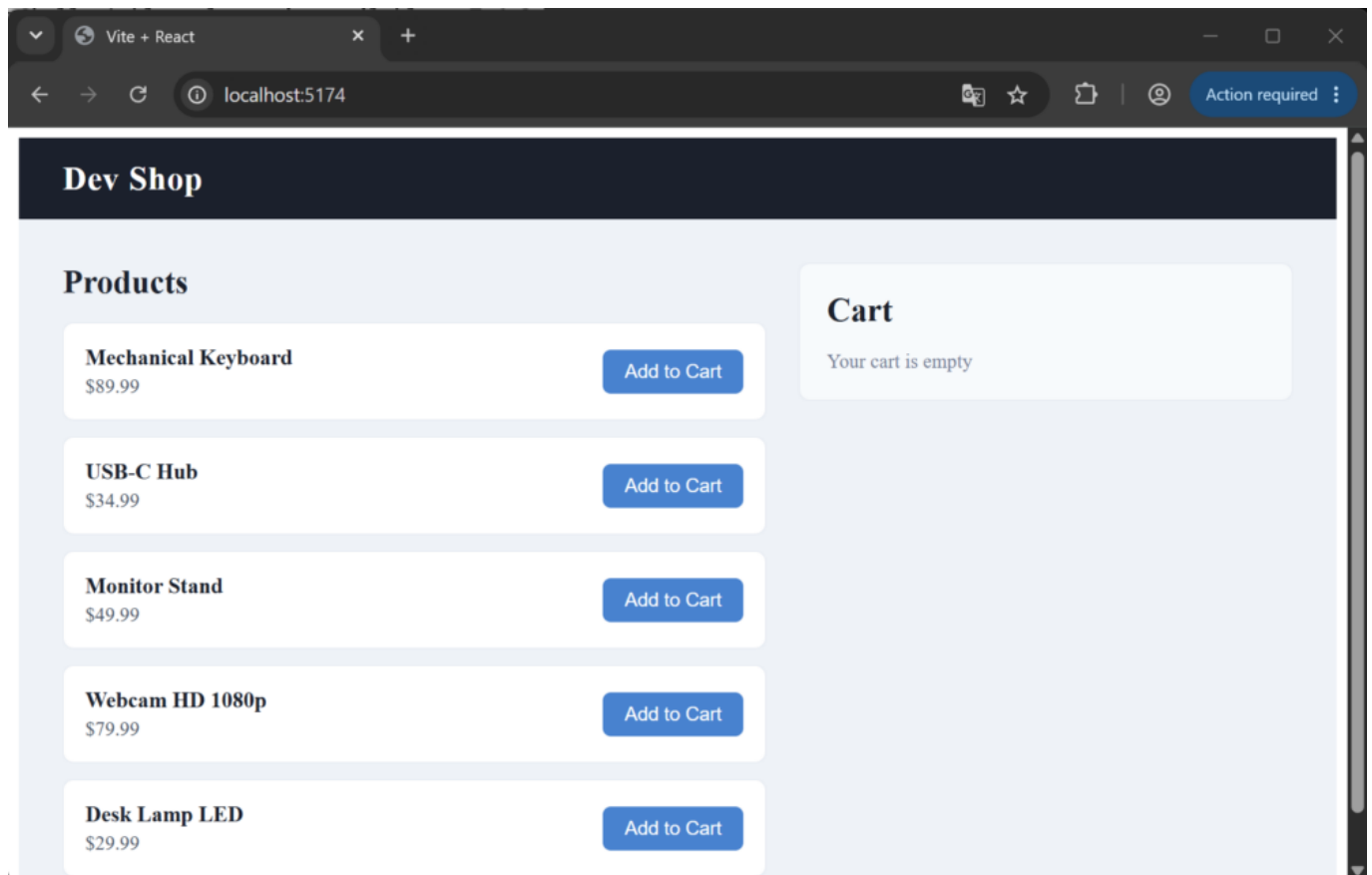
- **What is npm?** npm is the tool that downloads the project tools listed in package.json and runs commands for the app. npm is included in the official Node.js installer.

The terminal will show a local URL, usually:

`http://localhost:5173`

Open that URL in your browser. Keep the terminal running while you work on the app.

You should see the Dev Shop in your browser.



Step 4: Start Codex in the same folder

In the project terminal, start Codex:

```
codex
```

Codex should start inside the same `react_project` folder. This helps Codex see the files, docs, tests, and `AGENTS.md` for that specific app.

Step 5: Run the tests when the course asks you to verify work

These projects include tests. When you need to check whether the project still works, run:

```
npm run test
```

If you make changes with Codex, run the tests again before moving on.

If something does not work



- If npm is not found, install Node.js from <https://nodejs.org/download> and try again.
- If npm install fails, check that your terminal is inside the Shopping Cart App/react_project folder.
- If the browser does not show the app, use the exact local URL printed in your terminal.
- If Codex cannot find the files, start Codex from the same react_project folder.



Automated verification is one of the most valuable habits in AI-assisted development. In this lesson, we give Codex an objective way to check its own changes, shorten the correction cycle, and return work that has already passed the project's available checks.

Why plausible code is not enough

Code can look correct, compile successfully, and still contain subtle bugs. If we accept a confident explanation as proof, those quiet breakages may surface hours or days later. Manual checking also creates a delayed feedback loop: we leave Codex, test the result ourselves, describe any failure, and wait for another revision.

Automated verification addresses three related problems:

- **Quiet breakages:** tests and other checks can expose behavior that looks reasonable but is incorrect.
- **Slow feedback:** Codex receives error output immediately instead of waiting for us to reproduce and explain a problem.
- **Confidence without proof:** a passing command provides concrete evidence that complements Codex's explanation.

Establish a verified baseline

Before requesting a feature, we begin from a known state. For the Dev Shop project, we initialize a Git repository, stage the existing files, and create an initial commit:

```
git init && git add . && git commit -m "Initial commit"
```

We then run the existing test suite:

```
npm run test
```

A green baseline tells us the project works before Codex changes it. If a later check fails, we can connect that failure to the new work instead of wondering whether the problem already existed.

Ask Codex to verify every change

Our example task adds a cart discount. The important part of the request is not only the feature specification, but also the explicit completion condition:

```
Add a discount system to the cart. If the cart total exceeds $100,  
apply a 10% discount automatically and show the discounted total.  
Display the original total and the discount amount separately.
```

```
After making your changes, run npm run test and show me the results  
before considering this task complete.
```

Codex first explores the component structure and existing tests. It identifies `Cart.jsx` as the right place for the derived discount values, updates the cart summary and styling, and adds focused test cases on both sides of the \$100 threshold.

Implement and test the discount

The cart computes the discount from the current total during rendering. Totals above \$100 receive a 10% reduction, while lower totals keep a discount amount of zero:

```
const total = cartItems.reduce(
  (sum, item) => sum + item.price * item.quantity,
  0
);
const discountAmount = total > 100 ? total * 0.1 : 0;
const discountedTotal = total - discountAmount;
```

The interface displays the original total, conditionally displays the discount, and always displays the final total:

```
<p className={styles.summaryRow}>
  Original total: <strong>${total.toFixed(2)}</strong>
</p>
{discountAmount > 0 && (
  <p className={styles.summaryRow}>
    Discount: <strong>-${discountAmount.toFixed(2)}</strong>
  </p>
)}
<p className={styles.total}>
  Total: <strong>${discountedTotal.toFixed(2)}</strong>
</p>
```

The new tests confirm that the discount row stays hidden at or below \$100 and that the correct values appear above the threshold:

```
it('does not show a discount when the cart total is $100 or less', async () => {
  const { user } = setup();
  await user.click(screen.getAllByRole('button', { name: /add to cart/i })[0]);

  expect(screen.getByText('Original total:')).toBeInTheDocument();
  expect(screen.getByText('Total:')).toBeInTheDocument();
  expect(screen.queryByText('Discount:')).not.toBeInTheDocument();
});

it('applies a 10% discount when the cart total exceeds $100', async () => {
  const { user } = setup();
  const addButtons = screen.getAllByRole('button', { name: /add to cart/i });
  await user.click(addButtons[0]);
  await user.click(addButtons[1]);

  expect(screen.getByText('$124.98')).toBeInTheDocument();
  expect(screen.getByText('- $12.50')).toBeInTheDocument();
  expect(screen.getByText('$112.48')).toBeInTheDocument();
});
```



Codex runs the suite after implementing the feature and reports 12 passing tests. This result turns a plausible implementation into one supported by objective evidence.

```
Implemented the cart discount system.
```

```
Changed src/components/Cart.jsx:5 to derive:
```

- original total
- 10% discount when total exceeds `$100`
- discounted final total

```
Added styling in src/components/Cart.module.css:45 and added coverage in src/App.test.  
and no-discount cases.
```

```
Test results:
```

```
> react_project@0.0.0 test  
> vitest run
```

```
Test Files  1 passed (1)  
Tests       12 passed (12)  
Duration    939ms
```

Check the visible behavior

The running application provides an additional behavioral check. A total below the threshold receives no discount. Once the cart exceeds \$100, the interface displays the original total, the 10% reduction, and the discounted final total.

Mechanical Keyboard		<button>Remove</button>
\$89.99 × 4 = \$359.96		
Original total:		\$359.96
Discount:		-\$36.00
Total:		\$323.96

Shorten the correction loop

Without automated checks, we must inspect Codex's output, run the application, find a problem, return to Codex, and explain what failed. With verification in the task, Codex runs the check immediately after writing code. If the check fails, it can read the error output, diagnose the issue, revise the implementation, and rerun the command within the same session.

This does not remove our responsibility to review the work. It changes our role from performing the

first basic verification to reviewing a complete solution that already has evidence of functional correctness.

Make verification persistent with AGENTS.md

Repeating the verification requirement in every prompt works, but a project-level rule is more reliable. We can place the instruction in AGENTS.md so Codex receives it as persistent project context:

```
## Critical Rules
```

```
6. **ALWAYS run npm run test after making any changes to component  
   logic or state management. Do not consider a task complete  
   until all tests pass.**
```

The rule makes verification a standard completion requirement instead of an optional final step. It should name the exact command and clearly state when Codex must run it.

Choose the best available verification command

A dedicated test suite provides the strongest check for functional behavior, but other commands can still give Codex useful feedback:

- npm run test checks expected behavior and catches logic regressions.
- npm run build catches compilation, syntax, and type-related failures.
- npm run lint detects style violations, suspicious patterns, and convention problems.
- npm run dev provides a minimum check that the development server starts without immediately crashing.

We should choose the strongest relevant command that the project already supports. For important changes, combining several checks can provide broader coverage.

Commit verified work

After the implementation and its checks are complete, we stage and commit the verified changes:

```
git add . && git commit -m "feat: add 10% discount for orders over $100 - all tests p  
assing"
```

Automated verification is most effective when it becomes part of the project's normal definition of done. By giving Codex a concrete command, requiring successful results, and recording that rule in AGENTS.md, we create a faster feedback loop and a more dependable AI-assisted development workflow.

Real-World Workflows for AI Development with Codex - Verification Before Completion

1. Before Codex changes a working project, what gives you the clearest baseline?

- a. Run the existing checks and commit the current state
- b. Ask Codex whether the project looks stable
- c. Start the feature and inspect old behavior later
- d. Rely only on what appears in the interface

2. How do you make verification part of Codex's definition of done?

- a. Name the exact check and require it to pass
- b. Accept a confident summary as proof
- c. Run checks several tasks later
- d. Verify only when the interface breaks

Using the Weather Dashboard Starter Project

For the next part of the course, we will use the Weather Dashboard starter project.

Step 1: Open the Weather Dashboard project

Open this folder in Visual Studio Code:

```
? Starter Projects - Codex Workflows
  ? Weather Dashboard
    ? react_project ? open this folder
```

Make sure you open the react_project folder itself.

Step 2: Run the app locally

Open the VS Code terminal while you are inside the Weather Dashboard/react_project folder.

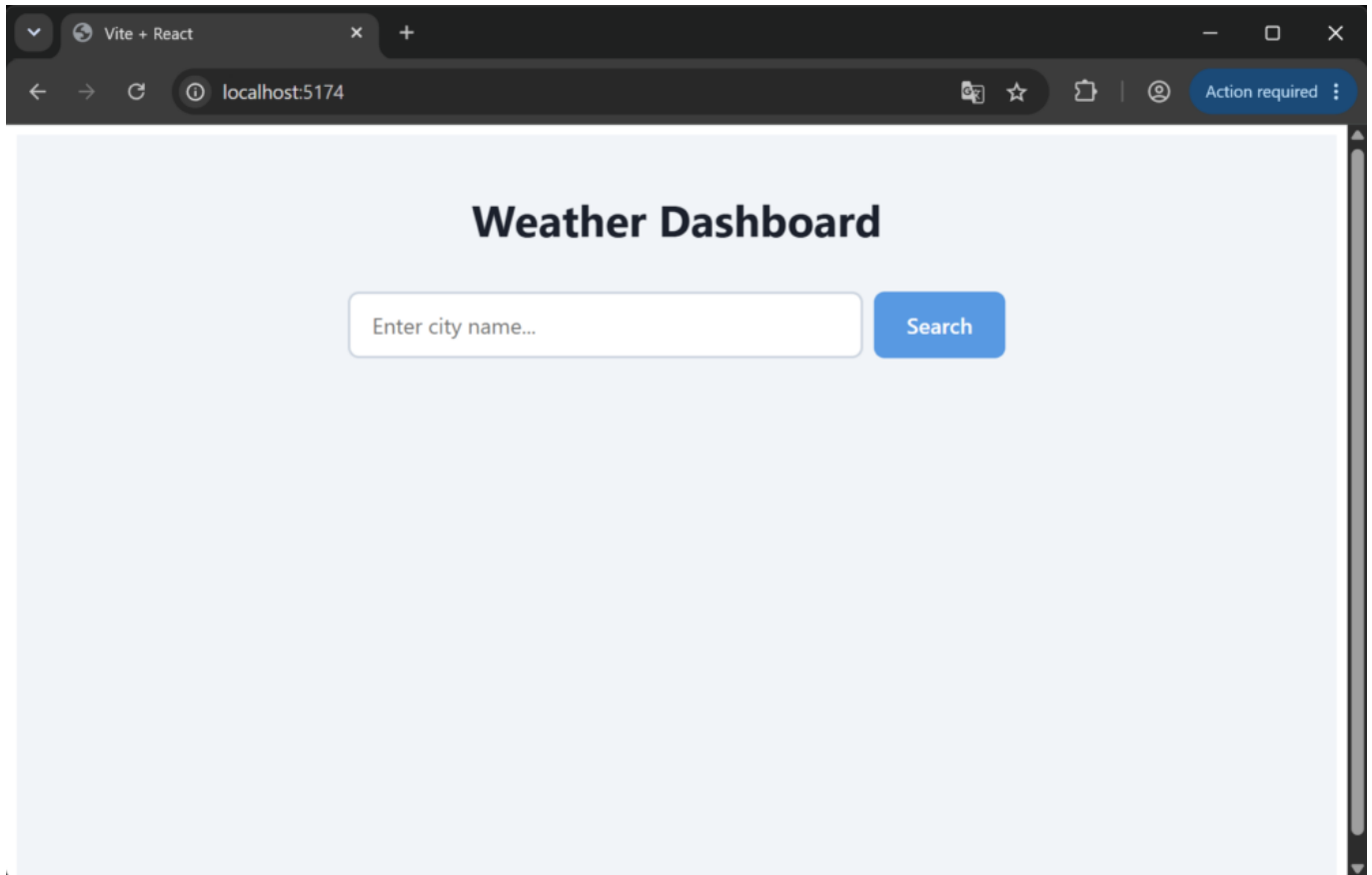
Then run:

```
npm install
npm run dev
```

Open the local URL shown in the terminal, usually:

```
http://localhost:5173
```

You should see the Weather Dashboard app in your browser.



Step 3: Start Codex in the same folder

In the project terminal, start Codex:

```
codex
```

Codex should start inside the same `react_project` folder so it can see the project files.

Step 4: Run the tests when needed

When the course asks you to verify work, run:

```
npm run test
```

If something does not work, first check that your terminal is inside the Weather Dashboard/`react_project` folder.



Effective debugging with Codex begins with precise evidence. Instead of translating an error into our own explanation, we can paste the raw output, state the desired result, and let Codex investigate the codebase. In this lesson, we will use that approach to repair two Weather Dashboard bugs, verify each fix, save the process as a reusable skill, and invoke that skill in a fresh session.

Give Codex the original evidence

Raw error output contains details that a summary can accidentally omit or distort. A parser error, failing assertion, stack location, and expected and received values all help Codex trace a failure to its source. Describing the problem from memory can introduce assumptions, while prescribing a specific fix can direct Codex away from the actual cause.

Our role is to provide the problem and desired outcome, then verify the result. We should avoid two common pitfalls:

- Do not replace the raw output with an interpretation of what we think happened.
- Do not micromanage the implementation before Codex has inspected the evidence.

Before making changes, we commit the current project state so that we have a clean baseline. We then run the test suite:

```
npm run test
```

The first failure combines a syntax problem with an incorrect Celsius-to-Fahrenheit formula in `src/Utils/temperatureUtils.js`. The malformed implementation is:

```
export const celsiusToFahrenheit = (celsius) => {  
  return (celsius * 1.5 + 32  
}
```

```
[plugin:vite:react-babel]
/Users/devisland/Desktop/learn_react/react_project/src/utils/temperatureUtils.js: Unexpected
token, expected ",", (3:0)
  4 |

/Users/devisland/Desktop/learn_react/react_project/src/utils/temperatureUtils.js:3:0
1 |   export const celsiusToFahrenheit = (celsius) => {
2 |     return (celsius * 1.5 + 32
3 |   }
  |   ^
4 |

at constructor (/Users/devisland/Desktop/learn_react/react_project/node_modules/@babel/parser/lib/index.js:365:19)
at JSXParserMixin.raise (/Users/devisland/Desktop/learn_react/react_project/node_modules/@babel/parser/lib/index.js:118:17)
at JSXParserMixin.unexpected (/Users/devisland/Desktop/learn_react/react_project/node_modules/@babel/parser/lib/index.js:130:17)
at JSXParserMixin.expect (/Users/devisland/Desktop/learn_react/react_project/node_modules/@babel/parser/lib/index.js:142:17)
at JSXParserMixin.parseParenAndDistinguishExpression (/Users/devisland/Desktop/learn_react/react_project/node_modules/@babel/parser/lib/index.js:165:17)
at JSXParserMixin.parseExprAtom (/Users/devisland/Desktop/learn_react/react_project/node_modules/@babel/parser/lib/index.js:235:17)
at JSXParserMixin.parseExprAtom (/Users/devisland/Desktop/learn_react/react_project/node_modules/@babel/parser/lib/index.js:235:17)
at JSXParserMixin.parseExprSubscripts (/Users/devisland/Desktop/learn_react/react_project/node_modules/@babel/parser/lib/index.js:250:17)
at JSXParserMixin.parseUpdate (/Users/devisland/Desktop/learn_react/react_project/node_modules/@babel/parser/lib/index.js:265:17)
at JSXParserMixin.parseMaybeUnary (/Users/devisland/Desktop/learn_react/react_project/node_modules/@babel/parser/lib/index.js:280:17)
at JSXParserMixin.parseMaybeUnaryOrPrivate (/Users/devisland/Desktop/learn_react/react_project/node_modules/@babel/parser/lib/index.js:295:17)
at JSXParserMixin.parseExprOps (/Users/devisland/Desktop/learn_react/react_project/node_modules/@babel/parser/lib/index.js:310:17)
at JSXParserMixin.parseMaybeConditional (/Users/devisland/Desktop/learn_react/react_project/node_modules/@babel/parser/lib/index.js:325:17)
at JSXParserMixin.parseMaybeAssign (/Users/devisland/Desktop/learn_react/react_project/node_modules/@babel/parser/lib/index.js:340:17)
at JSXParserMixin.parseExpressionBase (/Users/devisland/Desktop/learn_react/react_project/node_modules/@babel/parser/lib/index.js:355:17)
at /Users/devisland/Desktop/learn_react/react_project/node_modules/@babel/parser/lib/index.js:10780:39
at JSXParserMixin.allowInAnd (/Users/devisland/Desktop/learn_react/react_project/node_modules/@babel/parser/lib/index.js:10780:39)
at JSXParserMixin.parseExpression (/Users/devisland/Desktop/learn_react/react_project/node_modules/@babel/parser/lib/index.js:10780:39)
at JSXParserMixin.parseReturnStatement (/Users/devisland/Desktop/learn_react/react_project/node_modules/@babel/parser/lib/index.js:10780:39)
at JSXParserMixin.parseStatementContent (/Users/devisland/Desktop/learn_react/react_project/node_modules/@babel/parser/lib/index.js:10780:39)
```

We copy the complete test output, paste it into Codex, and add a concise instruction such as Fix the failing tests. Codex can now inspect the failing test and implementation without relying on our interpretation.

Verify the first fix

Codex identifies both the missing syntax and the incorrect multiplier. The corrected utility uses the standard conversion formula:

```
export const celsiusToFahrenheit = (celsius) => {
  // Convert Celsius to Fahrenheit using the formula:  $(C \times 9/5) + 32$ 
  return celsius * 1.8 + 32
}
```

After applying the focused edit, Codex runs `npm run test` and confirms that the test passes. We should also review the generated diff to make sure the change is limited to the intended file and formula. Automated tests provide strong evidence, but we also open the Weather Dashboard in the browser to confirm that the application works as expected.

Debug the city lookup

The next problem appears when a search for London returns a city-not-found message. This time, we



provide both the raw error and a short symptom description. That gives Codex the exact output plus the user action that produced it.

Codex traces the search from the input component to `getWeatherByCity`. The mock weather data uses lowercase keys, but the broken lookup converts the city name to uppercase:

```
return weatherData[cityName.toUpperCase()] ?? null
```

The focused fix normalizes the input to lowercase so that it matches the object keys:

```
// Returns weather data for a given city name, or null if not found
export const getWeatherByCity = (cityName) => {
  // Lookup is case-insensitive; convert input to lowercase to match object keys
  return weatherData[cityName.toLowerCase()] ?? null
}
```

We run the full test suite again, then repeat the London search in the browser. The application now returns the correct weather data.



Weather Dashboard

Search

London

Partly Cloudy

18°C 64°F

Wind Speed

12 km/h

With both issues resolved, we commit the verified fixes. Keeping this checkpoint separate produces a clear project history before we turn the process into a reusable capability.

Practice with a clear failing assertion

To repeat the workflow, we deliberately reintroduce an incorrect temperature multiplier and run npm run test. The test reports that converting 100°C returned 182 instead of the expected 212. This output gives Codex the failing file, test name, assertion, expected value, received value, and source location.



```
————Failed Tests 1————

FAIL  src/utils/temperatureUtils.test.js > celsiusToFahr
AssertionError: expected 182 to be 212 // Object.is equal

- Expected
+ Received

- 212
+ 182

> src/utils/temperatureUtils.test.js:6:38
   4| describe('celsiusToFahrenheit', () => {
   5|   it('converts 100°C to 212°F', () => {
   6|     expect(celsiusToFahrenheit(100)).toBe(212)
     |                                           ^
   7|   })
   8| })
```

We paste the raw failure and ask Codex to fix the failing test. Codex inspects `temperatureUtils.js` and its test, identifies the incorrect 1.5 multiplier, restores 1.8, and runs the tests. Once all tests pass, we review the diff and commit the patch with a clear message.

Save the debugging workflow as a skill

A workflow that we expect to repeat is worth saving. We ask Codex to turn the process it just performed into a reusable debug-and-verify skill. The skill records how to interpret raw failures, limit investigation to the relevant code, apply focused fixes, verify each change, review the diff, and commit only verified work.

name: debug-and-verify

description: Use when given raw failing test output for a local project and asked to debug it. Fix one failure at a time, run `npm run test` after each fix, review the diff, and commit only after confirming all tests pass.

Debug And Verify

Workflow

1. Read the failure output carefully.

- Identify the failing file, test name, assertion or parser error, expected value, received value, and stack location.
- Start with the first concrete failure. Do not guess at unrelated bugs.



2. Inspect the smallest relevant code surface.
 - Read the failing test and the implementation it exercises.
 - Use ``rg`` first for related symbols, messages, and imports.
 - Respect project instructions and avoid silently fixing unrelated intentional exercises.
3. Make one focused fix.
 - Keep edits limited to the files needed for the current failure.
 - Preserve existing style and project conventions.
4. Verify incrementally.
 - Run ``npm run test`` immediately after each fix.
 - If tests still fail, treat the new output as the next input and repeat from step 1.
1.
 - Continue until ``npm run test`` exits successfully.
5. Review the diff before committing.
 - Confirm the diff only contains the intended fixes.
 - Leave unrelated user changes untouched.
6. Commit the verified fix.
 - Check git status.
 - Stage only the files changed for this debugging task.
 - Commit only after tests pass.

Codex reviews its internal skill guidance and the project structure before creating `.codex/skills/debug-and-verify/SKILL.md`. Because the sandbox protects filesystem and Git operations, it requests approval before creating the directory, staging the file, and committing it. We review each proposed action and grant approval only when it matches the intended change.

After writing the skill, Codex validates its structure, checks repository status, runs the tests one final time, and reviews the staged diff. The commit should contain only the new skill file. These checks prevent unrelated changes from entering the commit.

Reuse the skill in a fresh session

To prove that the workflow is reusable, we introduce a similar error by changing the multiplier to 1.7, then start a new Codex session. After running the tests, we invoke the saved debug-and-verify skill.

Codex follows the stored procedure: it reads the failure, inspects the relevant utility and test, reviews the existing diff, and restores the correct formula:

```
return celsius * 1.8 + 32
```

It reruns the tests and checks Git status. In this case, no new commit is necessary because the fix restores the file to its already committed state, leaving the working tree clean.

Adopt the discover, save, and reuse loop

This workflow shifts our effort from prescribing solutions to confirming outcomes. Raw errors give Codex an accurate signal, focused instructions let it investigate autonomously, and tests plus diff



review keep us responsible for correctness.

1. Discover a reliable workflow by giving Codex raw evidence and a clear goal.
2. Verify the fix with tests, user-facing behavior, and a careful diff review.
3. Save the successful process as a skill when it will be useful again.
4. Reuse the skill in a fresh session and confirm that it produces the same disciplined result.

By following this loop, a one-time repair becomes a repeatable development capability. We reduce ambiguity, preserve project constraints, and spend more time validating results than manually diagnosing every failure.

Reviewing AI-generated code is not a formality. In this lesson, we will use Codex to inspect recent changes, compare quick and detailed review methods, apply a structured review checklist, and turn that process into a repeatable project rule.

Establish a review baseline

We begin with a clean commit history for the weather dashboard. A concise log lets us identify the starting point, the bug-fix commits, and the current state:

```
git log --oneline
```

A Git diff then reveals how the project changed. The reviewed history includes a new debug-and-verify skill and a correction to `getWeatherByCity`. The lookup now normalizes a city name to lowercase so it matches keys such as `london` and `tokyo`.

Create a change to review

To practice with uncommitted work, we ask Codex to add a loading spinner that matches the existing weather dashboard. Codex examines the project structure, adds loading state to `App.jsx`, creates `LoadingSpinner.jsx` and its CSS Module, and prevents an older delayed search from replacing a newer result.

Codex runs the automated tests and a production build. It also starts the Vite development server after receiving permission to retry the command. Before reviewing, we can use `git status --short` and `git diff` to confirm which files changed and inspect the implementation.

Run a quick review of uncommitted changes

Codex provides a built-in review command:

```
/review
```

From the review menu, we choose **Review uncommitted changes**. This asks Codex to scan staged, unstaged, and untracked work produced since the last commit.

```
• Implemented the loading spinner.

Changed:

- src/App.jsx: adds isLoading, simulates the mock fetch delay, hides stale results/errors while loading, and guards against older searches finishing after newer ones.
- src/components/LoadingSpinner.jsx: new accessible spinner component.
- src/components/LoadingSpinner.module.css: styling matched to the existing weather card/search UI.

Verified:

- npm run test passes
- npm run build passes

Dev server is running at http://127.0.0.1:5173/.

— Worked for 1m 19s —

Select a review preset

> 1. Review against a base branch (PR Style)
  2. Review uncommitted changes
  3. Review a commit
  4. Custom review instructions

Press enter to confirm or esc to go back
```

The review reports that the project builds, the existing test suite passes, and no discrete correctness issue was found. This is useful immediate feedback, but it is only a first filter. The command focuses on recent uncommitted work, not the entire codebase or the broader reasoning across older commits.

```
• Ran npm run build
  ↳ vite v7.3.1 building client environment for production...
    transforming...
    ... +5 lines (ctrl + t to view transcript)
    dist/assets/index-CNTZV3jZ.js 197.83 kB | gzip: 62.58 kB
    ✓ built in 492ms

• Explored
  ↳ Read package.json
    List .

• Ran npm run test
  ↳
    RUN v2.1.9 /Users/devisland/Desktop/learn_react/react_project
    ... +6 lines (ctrl + t to view transcript)
    Duration 171ms (transform 8ms, setup 0ms, collect 6ms, tests 1ms, environment 0ms, prepare 28ms)

<< Code review finished >>

• No discrete correctness issues were found in the current staged, unstaged, or untracked changes. The app builds and the existing test suite passes.
```

Request a detailed explanation of a Git diff

For a broader review, we ask Codex to compare a specific commit range and explain the result in plain English:

Run `git diff HEAD~5 HEAD` and explain every change in plain English.
For each file, explain what changed, why it changed, and what I should scrutinize.

Codex summarizes the affected files and the additions and deletions across the selected commits. It then explains the changes file by file, including the debugging skill, the city lookup correction, the loading state in `App.jsx`, the accessible spinner component, and its styling. Codex can also stop a development server that is no longer needed during the review.

```
.codex/skills/debug-and-verify/SKILL.md
What changed: Adds a new local Codex skill called debug-and-verify. It defines a workflow for handling failing
npm run test output: inspect the first failure, make one focused fix, rerun tests, review the diff, then commit
only verified changes.

Why it changed: This supports a repeatable debugging workflow for the course repo.

Scrutinise before accepting: Yes. This is project tooling, not app code. Check whether you want course starter
repos to include local agent instructions under .codex/. It may be useful for AI-assisted development lessons,
but it is metadata that changes how Codex behaves in this repo.

src/data/weatherData.js
What changed: getWeatherByCity now looks up weatherData[cityName.toLowerCase()] instead of
weatherData[cityName.toUpperCase()].

Why it changed: The mock data keys are lowercase, like london, tokyo, and new york. Uppercasing the input made
valid searches fail. Lowercasing makes the documented case-insensitive search behavior work.

Scrutinise before accepting: Yes, because AGENTS.md says broken behaviors are intentional student exercises.
Accept this only if this specific bug was meant to be fixed in this branch.

src/App.jsx
What changed: Adds isLoading state and a latestSearchId ref. Search now clears the old result/error, shows a
loading spinner, waits 600ms, then reads from mock weather data. The render path now shows LoadingSpinner while
loading and only shows errors/results once loading is finished.
```

Choose the right review method

The two approaches serve complementary purposes:

- **/review:** fast and automatic, focused on staged, unstaged, and untracked changes. Use it immediately after Codex completes a task to catch obvious problems.
- **Git diff with an explanation:** slower and more thorough, focused on the reasoning across a chosen commit range. Use it to understand why files changed and to identify flawed logic or unintended side effects.

A quick review provides a useful surface-level check. A detailed diff explanation helps us understand the strategy behind a complex feature or bug fix.

Review like a senior engineer

We should treat AI-generated changes like a pull request from a junior developer. Instead of accepting a passing test suite as sufficient evidence, we ask three questions:

1. **Did it touch the right things?** Confirm that every modified file was expected. Investigate unrelated or surprising changes.
2. **Does the approach make sense?** Understand the reasoning and verify that the implementation fixes the root cause, not only a symptom.
3. **Is anything missing?** Look for omitted tests, documentation, error handling, or other follow-up work required for a complete solution.

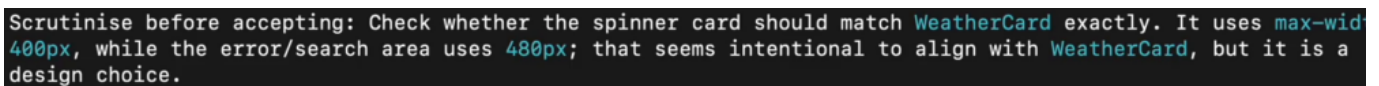
Act on missing test coverage

Applying this checklist reveals that `getWeatherByCity` lacks dedicated test coverage. We ask Codex to verify both a valid city and an invalid city. The utility uses a null fallback for missing data:

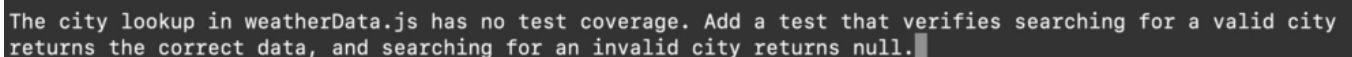
```
export const getWeatherByCity = (cityName) => {  
  return weatherData[cityName.toLowerCase()] ?? null  
}
```

The accompanying tests define the intended contract:

```
import { describe, expect, it } from 'vitest'  
import { getWeatherByCity } from '../weatherData'  
  
describe('getWeatherByCity', () => {  
  it('returns weather data for a valid city', () => {  
    expect(getWeatherByCity('London')).toEqual({  
      city: 'London',  
      temperature: 18,  
      windSpeed: 12,  
      condition: 'Partly Cloudy',  
    })  
  })  
  
  it('returns null for an invalid city', () => {  
    expect(getWeatherByCity('Atlantis')).toBeNull()  
  })  
})
```



Scrutinise before accepting: Check whether the spinner card should match `WeatherCard` exactly. It uses `max-width: 400px`, while the error/search area uses `480px`; that seems intentional to align with `WeatherCard`, but it is a design choice.



The city lookup in `weatherData.js` has no test coverage. Add a test that verifies searching for a valid city returns the correct data, and searching for an invalid city returns null.

After reviewing the proposed diff, we run the test suite and confirm that all tests pass. We then stage only the weather data utility and its new test so the commit remains focused. If Git reports an index-lock issue, Codex may need command approval before it can complete the commit. Once resolved, the working tree should be clean.

Make diff review a project rule

A reliable workflow should not depend on remembering every step manually. We add a critical rule to `AGENTS.md` so future Codex sessions repeat the review process before the final commit:

- Before the final commit of any session, run `/review` and then `git diff` to summarise every change made in plain English.

Finally, we commit the `AGENTS.md` update. This gives the project a clean, verified history and turns diff review into a consistent habit.

By combining a fast `/review`, a reasoned Git diff explanation, a three-question manual checklist, and



a persistent project rule, we can review AI-generated code with the same care we apply to any human contribution.

Real-World Workflows for AI Development with Codex - Debug and Review Changes

1. A test fails with a detailed stack trace. What should you give Codex first?

- a. The raw failure and the desired outcome
- b. Your guess about the exact code change
- c. A shortened error with technical details removed
- d. A request to rewrite the entire feature

2. /review finds no obvious issue in a complex change. What should you do next?

- a. Inspect the Git diff and scrutinize each changed file
- b. Commit immediately because the review passed
- c. Review only the files Codex mentions
- d. Replace the automated tests with manual testing

AI coding tools can move quickly, but speed does not remove our responsibility for critical code. In this lesson, we will identify the changes that always require human review, understand why automated tests are not enough for high-stakes paths, recognize Codex's blind spots, and establish practical checkpoints in our workflow.

Why tests are not enough

A comprehensive, passing test suite is valuable, but it only validates scenarios we anticipated when writing the tests. It cannot account for every unknown problem, subtle vulnerability, race condition, malicious input, or complex edge case.

This limitation is most dangerous in high-stakes code. Tests may confirm that expected behavior works under normal conditions, while a serious flaw remains outside the tested cases. Human experience and intuition provide a final checkpoint that automated systems cannot replace.

Code that requires mandatory human review

We do not need to apply the same level of scrutiny to every change. Instead, we should establish a non-negotiable review list for the areas where a mistake can have severe consequences:

- **Authentication and authorization:** Login, user access, and session management code can allow unauthorized access when implemented incorrectly.
- **Payment and billing logic:** Charging, refunding, and price-calculation code can create immediate financial consequences for users and the business.
- **Production data mutations:** Operations that write, update, or delete live database records need close review, especially when a change is difficult or impossible to reverse.
- **Destructive operations:** File deletion, database removal, and cache purging should receive line-by-line review before execution, particularly when Codex generated the command or code.
- **Access control and permissions:** Role checks, API guards, and related rules determine what authenticated users can do. Errors can expose sensitive data or enable unauthorized actions.
- **Sensitive third-party integrations:** Webhooks and APIs that handle personal or regulated data create a large surface area for mistakes and compliance problems.

Focus on external boundaries

These categories share a common pattern: they sit at the boundary between our application and something external. That external system might represent a user's identity, money moving through a payment gateway, sensitive data in a database, or the production environment itself.

Mistakes at these boundaries are often the most expensive because their consequences extend beyond the codebase. A typical component bug might temporarily break a user interface. A boundary bug can expose private data, charge an incorrect amount, or destroy records that cannot be recovered.

Understand Codex's blind spots

Codex is effective at producing happy-path implementations, but it may not anticipate every edge case, race condition, or adversarial input. These are precisely the situations where human experience matters most.

Codex also does not automatically understand our unique threat model or detailed business access rules. It can generate a permission check, but only we can confirm whether that check matches the project's actual security policies and user roles.



Finally, Codex does not have a complete view of the live production environment. It may not know the scale of the data, real user behavior, or every downstream dependency. A change that appears isolated can still cause cascading issues when deployed. We must apply our broader system knowledge before accepting it.

Build checkpoints into the workflow

We can make human oversight proactive by adding a rule to AGENTS.md that identifies critical files and folders. For example, the rule can require Codex to explicitly flag proposed changes to src/auth/, src/payments/, or database migration files for human review before committing them. This makes caution part of the project's normal AI behavior.

We should also use a written checklist before committing changes in high-stakes areas. Memory is unreliable when the stakes are high, while a checklist makes the required review consistent. In a later lesson, we will explore how to turn such checklists into custom commands.

Slow down for critical changes

When reviewing a high-stakes change proposed by Codex, we should deliberately reduce our pace. Instead of scanning it like a minor visual adjustment, we treat it like a legal contract and examine every line carefully. This conscious pause gives us space to identify dangerous assumptions, missing safeguards, and unintended effects.

The guiding principle is simple: **the faster Codex moves, the more carefully we read the high-stakes parts**. AI can accelerate development, but increased velocity requires greater human diligence where users, money, data, permissions, and production systems are involved.

By defining mandatory review categories, focusing on external boundaries, accounting for AI blind spots, adding AGENTS.md rules, using written checklists, and slowing down during critical reviews, we remain responsible guardians of sensitive systems while benefiting from Codex's speed.



Using the Finance Tracker Starter Project

For the next part of the course, we will use the Finance Tracker starter project.

Step 1: Open the Finance Tracker project

Open this folder in Visual Studio Code:

```
? Starter Projects - Codex Workflows
  ? Finance Tracker
    ? react_project ? open this folder
```

Make sure you open the react_project folder itself.

Step 2: Run the app locally

Open the VS Code terminal while you are inside the Finance Tracker/react_project folder.

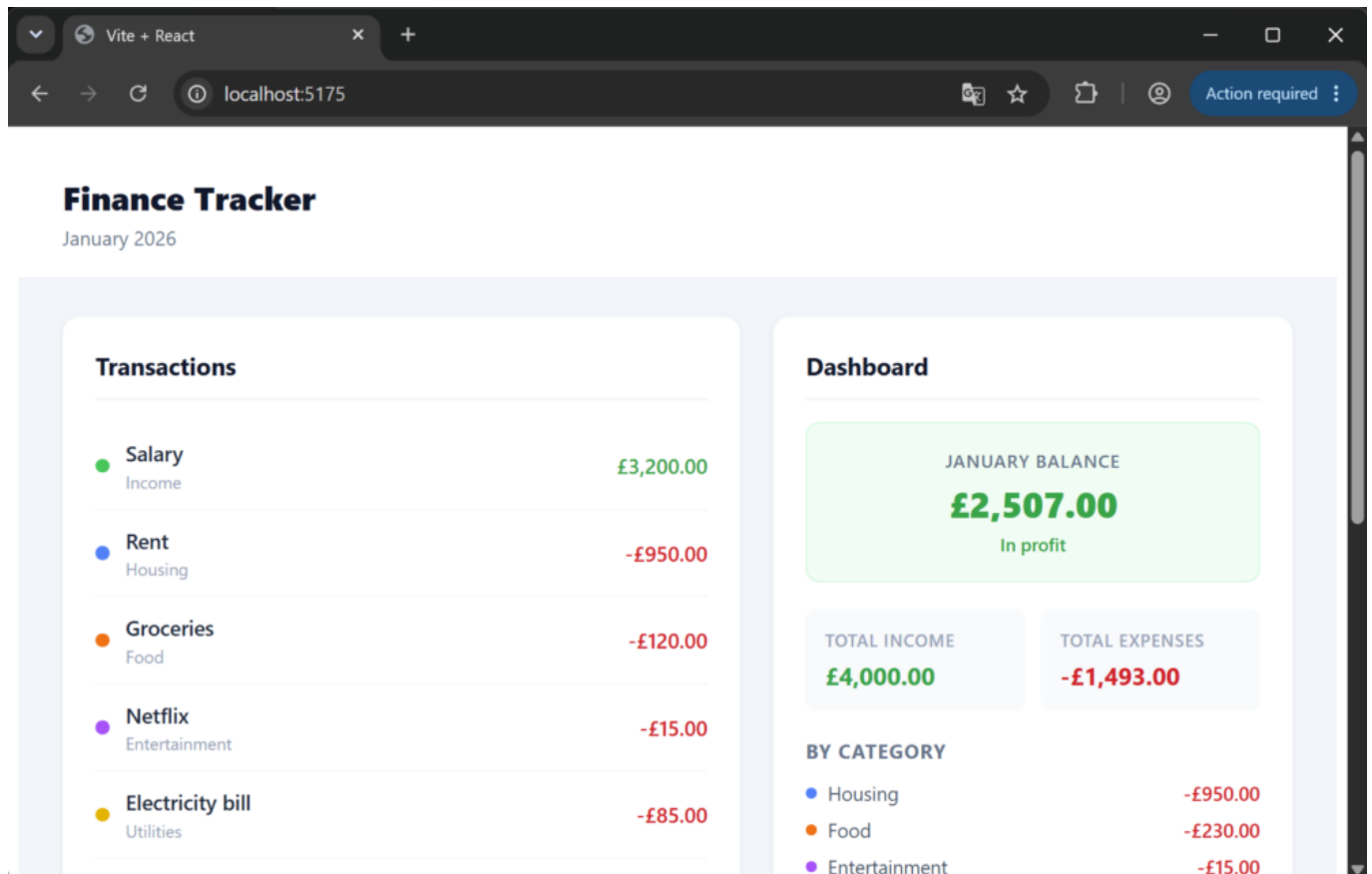
Then run:

```
npm install
npm run dev
```

Open the local URL shown in the terminal, usually:

```
http://localhost:5173
```

You should see the Finance Tracker app in your browser.



Step 3: Start Codex in the same folder

In the project terminal, start Codex:

```
codex
```

Codex should start inside the same `react_project` folder so it can see the project files.

Step 4: Run the tests when needed

When the course asks you to verify work, run:

```
npm run test
```

If something does not work, first check that your terminal is inside the `Finance Tracker/react_project` folder.



Subagents let us delegate a focused research task to a separate AI instance while keeping the main Codex session available for implementation. In this lesson, we will use a subagent to explore a finance tracker codebase, turn its findings into a focused implementation prompt, verify the resulting feature, and decide when delegation is worthwhile.

How subagents preserve context

A subagent is a secondary AI instance created from the main session. We give it one specific, self-contained job, such as researching a library, tracing an API, or exploring how a feature works across a codebase. It performs that work in its own context window and returns a concise summary when finished.

This separation protects the main session's working memory. Direct investigation can load many files and intermediate details into the primary context before implementation begins. A subagent keeps that exploratory material isolated, so the main session receives only the findings needed to move forward.

We begin by opening the personal finance tracker project, initializing Git, and confirming that the repository has a clean initial commit. We then launch Codex from the project directory and use the `/status` command to establish a baseline for the model, permissions, directory, usage limits, and context consumption.

```
Visit https://chatgpt.com/codex/settings/usage for up-to-date
information on rate limits and credits
```

```
Model:          gpt-5.5 (reasoning medium, summaries auto)
Directory:      ~/Desktop/learn_react/react_project
Permissions:    Workspace (Ask for approval)
Agents.md:      AGENTS.md
Account:        [REDACTED] (Plus)
Collaboration mode: Default
Session:        019ed6ee-5d76-7172-a2e7-865e75eeadd4
```

```
5h limit:       [████████████████████] 99% left (resets 00:53 on 18 Jun)
Weekly limit:   [████████████████████] 98% left (resets 12:03 on 23 Jun)
```

Delegate a deliberate research task

For the demonstration, we need to understand how category calculations work and what must change to add budget alerts. Researching directly would require the main session to inspect the relevant files, follow the calculation flow, and retain those details alongside the later implementation work.

Instead, we explicitly ask Codex to use a subagent. The prompt defines the research goal, identifies the questions to answer, requests a concise report, and states that the main session must not explore the files. This last instruction makes the context-preservation objective unambiguous.

```
> Use a subagent to explore the codebase and investigate how category calculations currently work – which files
  handle them, where the logic lives, and what would need to change to add a budget alert system that warns when
  a category exceeds a set limit. The subagent should report back a concise summary. Do not explore the files in
  this main session.
```

Codex spawns an isolated subagent to inspect the project. The subagent traces the category logic, identifies the files involved, and recommends an approach for integrating the budget alert system.



While that investigation runs, the main session remains focused and does not absorb the codebase exploration.

When the subagent completes its task, it returns a compact report. In this project, the report identifies the transaction service as the main location for filtering and aggregation, then outlines the files and responsibilities that should change for the new feature.

```
• Finished waiting
  ↳ 019ed6f8-8e9d-77d3-bc5d-8416903c36ad: Completed - No files modified. **Category Calculation Flow** - [src/services/transactionService.js](/Users/devisland/Desktop/learn_react/react_project/src/services/transactionService.js:7) - `getTransactionsByCategory(category)` filters hardcoded tr...

• Subagent report received. No files were modified.

Category calculations currently live mainly in src/services/transactionService.js:

- getTransactionsByCategory(category) filters transactions by category.
- getSummaryByCategory() aggregates transaction totals by transaction.category and returns objects like { category, total }.
```

Running `/status` again confirms the result: extensive exploration occurred in the subagent's context window, while the primary context gauge barely changed. The main session is therefore still well positioned to handle planning, implementation, and verification.

```
>_ OpenAI Codex (v0.140.0)
```

```
Visit https://chatgpt.com/codex/settings/usage for up-to-date
information on rate limits and credits
```

```
Model:          gpt-5.5 (reasoning medium, summaries auto)
Directory:      ~/Desktop/learn_react/react_project
Permissions:    Workspace (Ask for approval)
Agents.md:     AGENTS.md
Account:        ██████████ (Plus)
Collaboration mode: Default
Session:       019ed6ee-5d76-7172-a2e7-865e75eeadd4
```

```
Context window: 98% left (17.6K used / 258K)
5h limit:       [██████████] 98% left (resets 01:04 on 18 Jun)
Weekly limit:   [██████████] 98% left (resets 12:03 on 23 Jun)
```

Control the subagent's focus and output

Effective delegation depends on a deliberate prompt. We can shape a subagent's behavior in several ways:

- **Define a precise scope:** State exactly what the subagent should investigate and what it should ignore.
- **Request a useful report format:** Ask for concise bullet points, a short paragraph, key files, or another format that supports the next decision.
- **Set an appropriate level of detail:** A broad report can summarize architecture, while a narrower report can trace a specific dependency or behavior.
- **Chain subagents for complex work:** One subagent's findings can become the input for another focused investigation while the main session coordinates the overall workflow.

The goal is not merely to move work elsewhere. The goal is to receive a small, actionable result that



lets the main session proceed without repeating the exploration.

Turn research into a focused implementation

Once the research is available, we convert it into a concrete implementation prompt. For the finance tracker, the prompt requests hardcoded monthly budget limits, requires the service layer to perform calculations, keeps the category summary component presentational, and asks Codex to run the test suite after making changes.

```
› Based on the subagent's findings, add a budget alert system to the finance tracker. Each category should have a hardcoded monthly budget limit. If actual spending exceeds the limit, show a warning indicator on the CategorySummary component. Use the existing services layer for the calculation – do not add calculation logic inside components. Run npm run test:run after making changes.
```

Because the subagent already identified the architecture and relevant files, Codex can plan the change without another broad investigation. The implementation adds monthly budget data, calculates spending and alert metadata in the transaction service, passes the prepared results through the application, and displays an over-budget warning in the category summary.

Verification remains essential. Codex runs the test suite and finds an expectation mismatch for the Utilities category because its January spending exceeds the new configured limit. It updates the service test to match the budget data, reruns the suite, and confirms that all 14 tests pass. A lint check also completes with zero errors. This demonstrates the complete workflow: delegated exploration, focused implementation, and verified results.

Choose between delegation and direct investigation

Subagents are valuable, but they are not necessary for every question. We can use the following framework to choose the more efficient approach.

Use a subagent when:

- The research is broad and requires reading many files or tracing a complex dependency tree.
- The exploration would consume significant context that the main session needs for implementation.
- A synthesized summary is sufficient to make the next decision.
- We are onboarding to an unfamiliar codebase and need a high-level map before starting focused work.

Investigate directly when:

- A question can be answered by one or two files.
- The expected context cost is small.
- We need to inspect the exact code and its nuances before making a critical decision.
- Research and implementation are tightly coupled in a rapid read, decide, and edit loop.

Delegation has some setup and reporting overhead. For a small, self-contained question, direct investigation is often faster. For a broad research task, isolating the work usually preserves more capacity than the delegation costs.

Commit the verified result

After implementation and verification, we ask Codex to commit the changes with a clear message that reflects the budget alert feature and the role of subagent research. Codex checks the repository, notices that the new monthly budget file must be tracked, and requests permission



before staging and committing the files.

Once the commands are approved, Codex stages the changes, creates the commit, and checks the repository again. The final output confirms the commit and a clean working tree.

```
• The changes are staged now. I'm creating the commit with the exact message you provided.
✓ You approved codex to run git commit -m "add category budget alert system using subagent research" this time
• Ran git commit -m "add category budget alert system using subagent research"
  ↳ [main ffda41e] add category budget alert system using subagent research
    6 files changed, 108 insertions(+), 5 deletions(-)
    create mode 100644 src/data/monthlyBudgets.js

• The commit succeeded. I'm checking the working tree once so I can report whether anything remains uncommitted.
• Ran git status --short
  ↳ (no output)
```

Key takeaways

- Subagents are isolated Codex instances with their own context windows.
- Delegating broad research protects the main session's capacity for implementation.
- Clear scope, explicit exclusions, and a requested report format produce more useful results.
- A subagent's concise findings can support a highly focused implementation prompt.
- Direct investigation remains the better choice for small questions, critical code review, or tightly coupled implementation work.

Real-World Workflows for AI Development with Codex - Keep Human Control

1. Codex changes authentication code and all tests pass. What remains essential?

- a. Review the high-stakes change carefully yourself
- b. Trust the tests because authentication is covered
- c. Ask Codex to approve its own implementation
- d. Skip review if the diff is short

2. You need broad codebase research without filling the main session's context. What fits best?

- a. Give a subagent a precise research task and report format
- b. Ask the main session to read every file
- c. Start implementing before investigating
- d. Request a broad report with no scope limits



Parallel Codex sessions let us work on independent parts of a project at the same time. This can shorten development time, but only when each session has a clearly defined area of ownership. In this lesson, we will run two Codex instances against a finance tracker project, assign one to the interface and one to the service layer, verify both results, and decide when parallel work is safe.

Understand clean ownership boundaries

Each parallel session must own a distinct set of files. If two sessions edit the same file, their changes can collide, producing broken code and difficult debugging. Before starting, we should be able to state exactly which files or directories each session may modify.

Our two tasks have a useful natural separation:

- The UI session adds a description search field in `src/components/TransactionList.jsx` and its component stylesheet.
- The data session adds spending calculations and tests in `src/services/`.

Because the component and service directories do not overlap, the sessions can proceed independently. We should also create a clean Git checkpoint before either session begins:

```
git add . && git commit -m "pre-parallel session checkpoint"
```

This baseline gives us a known good state that we can restore if either task produces an unexpected result.

Launch the two Codex sessions

Open two terminals in the same project and run codex in each one. The first instance will own the UI task, while the second will own the service task. We do not need to wait for the first session to finish before prompting the second one.

The first prompt should describe the required behavior and make the file boundary explicit:

Add a search bar above the transaction list in `TransactionList.jsx`.
As the user types, filter the visible transactions to show only those whose description contains the search text, case insensitive. If the search field is empty, show all transactions.

You are only allowed to modify files inside `src/components/`.
Do not touch `src/services/`, `src/utils/`, or `src/data/`.

Run `npm run test:run` after making your changes.

```
>_ OpenAI Codex (v0.141.0)
```

```
model:      gpt-5.5   /model to change  
directory:  ~/Desktop/learn_react/react_project
```

Tip: Use `/fork` to branch the current chat into a new thread.

```
> Add a search bar above the transaction list in TransactionList  
As the user types, filter the visible transactions to show only  
those whose description contains the search text – case insensitive.  
If the search field is empty, show all transactions.
```

You are only allowed to modify files inside `src/components/`.
Do not touch `src/services/`, `src/utils/`, or `src/data/`.

Run `npm run test:run` after making your changes.

The boundary is as important as the feature request. It keeps the filtering implementation and styles inside the component layer and prevents this session from interfering with the service work.

Implement the component search

Codex inspects the existing component structure, keeps the search state local to `TransactionList.jsx`, and adds scoped styles in `TransactionList.module.css`. The central filtering logic is:

```
const [searchText, setSearchText] = useState('');  
const normalizedSearchText = searchText.trim().toLowerCase();  
  
const visibleTransactions = normalizedSearchText  
  ? transactions.filter((transaction) =>  
    transaction.description.toLowerCase().includes(normalizedSearchText),  
  )  
  : transactions;
```

The controlled search input updates that state and the list renders `visibleTransactions`:

```
<label className={styles.searchLabel} htmlFor="transaction-search">  
  Search transactions  
</label>  
<input  
  className={styles.searchInput}  
  id="transaction-search"  
  type="search"/>
```



```
value={searchText}
onChange={(event) => setSearchText(event.target.value)}
placeholder="Search by description"
/>
```

Trimming and lowercasing the query makes the match case insensitive and avoids treating whitespace as a meaningful search. When the normalized query is empty, every transaction remains visible. After adding the component and CSS changes, the session runs `npm run test:run` to verify the project.

Assign the independent service task

While the UI session works, we can prompt the second Codex instance. This task adds two calculations and their tests, with ownership restricted to the service directory:

Add two new functions to `src/services/transactionService.js`:

1. `getTopSpendingCategory()`, which returns the category with the highest total expenses.
2. `getAverageDailySpend()`, which calculates average daily spending over a 31-day month and rounds the result to two decimal places.

Add tests for both functions in `transactionService.test.js`.

You are only allowed to modify files inside `src/services/`.
Do not touch `src/components/`, `src/utils/`, or `src/data/`.

Run `npm run test:run` after making your changes.



Add two new functions to `transactionService.js`:

1. `getTopSpendingCategory()` – returns the category name with the highest total expenses as a positive number.
2. `getAverageDailySpend()` – assumes a 31-day month, divides total expenses by 31, returns a positive number rounded to 2 decimal places.

Also add tests for both functions in `transactionService.test.js`.

You are only allowed to modify files inside `src/services/`. Do not touch `src/components/`, `src/utils/`, or `src/data/`.

Run `npm run test:run` after making your changes.

This second session can inspect the service implementation and its tests, but it must leave the component changes from the first session untouched.

Add service calculations and tests

The service uses the existing expense summary and total-expense functions. The new functions are:

```
export function getTopSpendingCategory() {
  const [topCategory] = getExpenseSummaryByCategory().sort(
    (a, b) => b.spent - a.spent,
  );

  return topCategory.category;
}

export function getAverageDailySpend() {
  return Number((Math.abs(getTotalExpenses()) / 31).toFixed(2));
}
```

`getTopSpendingCategory()` sorts expense summaries from highest to lowest spending and returns the first category. `getAverageDailySpend()` converts the negative expense total to a positive amount, divides it by 31, rounds to two decimal places, and converts the result back to a number.

The focused tests verify the results from the seeded transaction data:

```
it('getTopSpendingCategory returns the category with the highest total expenses', () => {
  expect(getTopSpendingCategory()).toBe('Housing');
});
```

```
it('getAverageDailySpend returns the average expense spend over 31 days', () => {  
  expect(getAverageDailySpend()).toBe(48.16);  
});
```

The expected top category is Housing, and the average daily spend is 48.16. Codex runs the full one-shot test command and confirms that all three test files and all 16 tests pass.

Verify boundaries and combine the work

Passing tests are necessary, but we should also verify that each session stayed inside its assigned territory. The service session checks the relevant diff and then runs `git status --short`. The status includes component files changed by the UI session and service files changed by the data session, while the service session's own summary lists only:

- `src/services/transactionService.js`
- `src/services/transactionService.test.js`

```
• Ran git status --short  
  L M src/components/TransactionList.jsx  
    M src/components/TransactionList.module.css  
    M src/services/transactionService.js  
    M src/services/transactionService.test.js
```

```
• Implemented the two new service functions and added tests in:  
  
- src/services/transactionService.js  
- src/services/transactionService.test.js  
  
getTopSpendingCategory() returns 'Housing', and getAverageDailySpend() returns 48.16  
  
Verification: npm run test:run passed with 3 test files and 16 tests passing
```

Once both sessions finish, run the full project test suite again to verify that the two sets of changes work together. We can then consolidate the completed work in one commit:

```
git add . && git commit -m "add search bar and new service functions - built in parallel"
```

Monitor for boundary violations

Parallel sessions are not a set-and-forget workflow. Even with a precise prompt, an agent may attempt to read or modify a forbidden file. Watch tool-use notifications for unexpected paths. If a session crosses its boundary, press Escape to stop it, restate the permitted directory, and then allow it to continue within the corrected scope.

This monitoring prevents one session from overwriting or invalidating work owned by another



session. The final test run and diff review provide a second layer of protection.

Decide when parallel work is worthwhile

Parallel sessions are most effective when tasks are independent and ownership is unambiguous. Good candidates include:

- Front-end and back-end features that live in separate directories.
- A refactor in one layer and a new feature in another layer.
- Tests for existing code while a separate feature is developed elsewhere.

We should keep work sequential when two tasks modify the same files or when one task depends on the result of another. For example, if a UI feature requires an API that does not exist yet, complete the API work first. A practical rule is simple: if we cannot draw a clear, unambiguous ownership boundary for every session, the coordination and collision risks outweigh the possible speed improvement.

By combining clean checkpoints, exclusive file ownership, active monitoring, test runs, and final diff review, we can use multiple Codex sessions as a safe AI development assembly line.



Codex slash commands provide quick control over an interactive session. They help us start or end sessions, manage context, inspect configuration, plan and review work, adjust permissions and models, and access more advanced workflows. In this lesson, we will examine the essential slash commands and compare interactive use with non-interactive automation through `codex exec`.

Start and end sessions

Use `/new` when we want a clean slate for an unrelated task. It clears the conversation history and starts a fresh session while keeping us inside Codex. This makes it suitable for switching between distinct pieces of work without leaving the interactive interface.

Use `/exit` when the current session is finished. It closes Codex and returns us to the terminal.

Manage the context window

`/compact` is a gentler alternative to `/new`. Instead of discarding the session, it compresses older conversation content into a concise summary. Codex preserves important information such as decisions, files that were touched, and the current task while freeing context space.

In the demonstrated finance tracker session, `/status` initially reports 16.8K tokens used. After running `/compact`, another `/status` check reports 6.22K tokens used. This confirms that the conversation was compacted while the current thread remained available. We should use `/compact` when the context gauge is becoming full but we still want to continue the same task. We should use `/new` when beginning a separate task or session.

Inspect status and plan work

The `/status` command gives us an immediate snapshot of the current session. Its output includes the active model, permissions, context usage, and approaching rate limits. We can run it whenever we need to confirm the environment or investigate why a session feels different from expected.

The `/plan` command switches Codex into read-only planning mode. Codex analyzes the request and proposes a strategy before making file changes. This is useful before significant or complex feature work because it lets us review the intended approach before implementation begins.

Review uncommitted changes

The `/review` command provides a structured critique of current uncommitted changes. It acts as a pre-commit quality check by looking for work that is incomplete, suspicious, or inconsistent. Its analysis is focused on the recent uncommitted changes rather than the entire project.

We should treat the output as informed advice from a colleague, not as a mandatory list of fixes. Running `/review` before an important commit can reveal issues that basic functional checks may miss while leaving the final decision with us.

Control permissions and models

The `/permissions` command lets us switch approval and sandbox modes within the current session. This gives us direct control over what Codex may do, which is especially important before sensitive operations such as writing to a database.

The `/model` command changes the underlying AI model without ending the session. We can use it to match the model to the complexity of the task or to optimize cost and efficiency while keeping the current conversation active.



Use advanced workflow commands

Codex includes several commands for reusable workflows, connected tools, and alternative development paths:

- `/skills` lists available skills and lets us invoke a reusable workflow by name.
- `/mcp` shows which MCP servers are connected to the current session and reports their status. We can check this before relying on an MCP-based tool.
- `/fork` clones the current session into a new thread. This preserves the original context while we explore an alternative solution or development path.
- `/init` analyzes the project and generates an initial `AGENTS.md` file. It is useful when starting a project or working in an existing project that does not yet have this project guidance.

Automate tasks with Codex Exec

Slash commands control an interactive Codex session. For automation, `codex exec` runs a natural-language instruction without opening the full interactive terminal interface. Codex performs the requested task and then exits, making this mode suitable for scripts and continuous integration pipelines.

A complete example is `codex exec "run the test suite and report any failures" --json --sandbox workspace-write --ephemeral`. The quoted text is the same kind of prompt we would enter interactively. The additional flags refine how the automated run behaves:

- `--json` produces structured, machine-readable output for scripts, reports, and pipelines.
- `--sandbox workspace-write` explicitly applies a sandbox policy that permits work within the workspace while maintaining defined safety boundaries.
- `--ephemeral` prevents the one-off session from being persisted to disk, which avoids adding isolated automated tasks to long-term session history.

Interactive slash commands are the right choice when we are actively collaborating with Codex and managing an ongoing session. `codex exec` is the right choice when a task should run non-interactively as part of a command-line script or CI workflow. Together, these tools let us move between direct development assistance and repeatable automation.

Real-World Workflows for AI Development with Codex - Coordinate Codex Sessions

1. Two independent tasks affect separate directories. What is the safest parallel approach?

- a. Assign exclusive file ownership and run final tests together
- b. Let both sessions edit any file they need
- c. Commit each change without checking the combined result
- d. Use parallel sessions even when one task depends on the other

2. Your context is filling up, but you want to continue the same task. Which command fits?

- a. /compact
- b. /new
- c. /exit
- d. /init

Hooks add event-driven automation to Codex workflows. Instead of waiting for us to request an action manually, a hook runs a shell command when a specific Codex lifecycle event occurs. In this lesson, we will configure project-level hooks, test a Stop hook, add session-start context, compare hooks with AGENTS.md and skills, and examine the current limitations of the feature.

Store hook configuration

Hooks use a dedicated hooks.json file rather than the general config.toml file. Project-specific hooks belong in .codex/hooks.json at the project root. We can also define global hooks in ~/.codex/hooks.json. Codex loads the global definitions alongside the project's hooks, allowing us to combine reusable personal automation with repository-specific safeguards.

Create the project-level file with:

```
mkdir -p .codex && touch .codex/hooks.json
```

Choose a lifecycle event

Each hook attaches an automatic command to a particular point in the Codex lifecycle:

- SessionStart fires when a session starts or resumes. It can run setup scripts, load context, or prepare the environment.
- PreToolUse fires before Codex runs a Bash command. It can perform checks, confirm permissions, or block a dangerous operation.
- PostToolUse fires after a Bash command completes. It is useful for tests, linters, and other validation checks.
- UserPromptSubmit fires whenever we submit a prompt. It can scan for sensitive information, enforce formatting, or attach metadata.
- Stop fires after Codex finishes its complete response, whether or not it used a tool. It is a suitable point for a comprehensive test suite or final integrity check.

The event determines when automation runs, so we should choose the narrowest lifecycle point that matches the intended safeguard or maintenance task.

Configure a Stop hook

We will begin with an observable Stop hook. Add this configuration to .codex/hooks.json:

```
{
  "hooks": {
    "Stop": [
      {
        "hooks": [
          {
            "type": "command",
            "command": "echo \"Stop hook fired at $(date)\" >> /tmp/codex-hooks.log"
          }
        ]
      }
    ]
  }
}
```

The outer hooks object groups events. The Stop array contains the event configuration, and its nested hooks array lists the actions to run. This action has the type command and appends a timestamped message to /tmp/codex-hooks.log. Redirecting the output to a file keeps the console clean while giving us a reliable way to verify the event.

Review and trust hooks

When Codex detects a new or changed hook, it asks us to review it. Hooks require special care because trusted commands can run outside the sandbox. We can inspect the hook, trust all hooks and continue, or continue without trusting them. The last option keeps the hooks from running in that session.

Hooks need review

1 hook is new or changed.

Hooks can run outside the sandbox after you trust

1. Review hooks

2. Trust all and continue

3. Continue without trusting (hooks won't run)

We should inspect every command before trusting it. A hook is persistent automation, so a small mistake or unsafe command can affect every matching event rather than a single interactive request.

Verify the Stop hook

To test the configuration, we start Codex and assign a normal development task: add `getLowestSpendingCategory()` to `src/services/transactionService.js`, return the category with the lowest total expenses, and run `npm run test:run`. Codex inspects the existing service and tests, then adds the helper by reusing `getExpenseSummaryByCategory()`:

```
export function getLowestSpendingCategory() {
  const [lowestCategory] = getExpenseSummaryByCategory().sort(
    (a, b) => a.spent - b.spent,
  );

  return lowestCategory.category;
}
```

The ascending sort places the category with the smallest spending total first. Codex then runs the requested tests and reports that all three test files and all 16 tests pass.

After the response is complete, inspect the hook log:

```
cat /tmp/codex-hooks.log
```



- Added `getLowestSpendingCategory` in `src/service` `getExpenseSummaryByCategory()` and returns the

Verification: `npm run test:run` passed: 3 test

```
> cat /tmp/codex-hooks.log
```

- I'll read that log directly and relay the outp

- **Explored**

- └ `Read` `codex-hooks.log`

-
- `/tmp/codex-hooks.log` contains:

Stop hook fired at Sat 20 Jun 2026 21:16:59 BS

The timestamped Stop hook fired entry confirms that the hook ran after Codex finished its response. This simple log-based check gives us a controlled starting point before replacing the command with more complex automation.

Inject context when a session begins

A `SessionStart` hook can automatically provide important context whenever Codex begins or resumes a session. To create one, add a `SessionStart` event to `hooks.json`, define a command such as `echo` that prints a project reminder, and set its matcher to `startup|resume`. The matcher ensures that the reminder appears for both a new session and a resumed session.

This approach is useful for information that should become visible at the start of every relevant interaction. It removes the need to paste the same reminder manually and makes session initialization consistent.



Place hooks in the automation stack

Codex offers three complementary ways to shape work:

- AGENTS.md provides always-on project rules and constraints. We do not invoke it manually.
- Skills package reusable, multi-step workflows that we invoke on demand.
- Hooks run shell commands automatically in response to lifecycle events.

AGENTS.md defines expected behavior, skills provide deliberate procedures, and hooks supply continuous event-driven checks or maintenance. Selecting the appropriate layer keeps automation understandable and prevents one mechanism from taking on responsibilities better handled by another.

Account for current limitations

The hook system has important boundaries. In the version demonstrated in this lesson, PreToolUse and PostToolUse respond only to Bash commands. They do not fire for file writes, web searches, or other internal Codex tools. Hooks are also unavailable on Windows, and the feature remains experimental. Configuration formats, event names, and behavior may change, so we should review the changelog when upgrading Codex.

Finally, commit hooks.json to the repository so the automation is version-controlled and shared with the team:

```
git add . && git commit -m "add hooks configuration"
```

With the configuration stored in the project, Codex picks it up automatically at the start of the next session. By choosing precise events, reviewing commands before trusting them, verifying their effects, and keeping the file under version control, we can use hooks as a reliable automation layer alongside project rules and reusable skills.

Choosing the right Codex model is only one part of an effective AI-assisted development workflow. We also need a reliable way to begin each session, a clear understanding of when Model Context Protocol (MCP) servers are useful, and habits that improve the project over time. Together, these practices help us balance capability, speed, cost, safety, and focus.

Match the model to the task

The default model is our everyday workhorse. It is the best starting point for most coding work, including implementing features, fixing bugs, and writing tests. When we are unsure which model to use, starting with the default keeps the workflow simple and cost-effective.

Reasoning-heavy models are better suited to intricate problems. We can switch to one when a task involves a significant architectural decision, a difficult investigation, or a multi-file refactor that requires deeper understanding. These models generally cost more and can take longer to respond, so we should reserve them for exchanges that genuinely need the additional reasoning.

Faster, lightweight models are useful for simpler and repetitive work across multiple files. Examples include adding comments or producing routine boilerplate. Using a lightweight model for these tasks avoids paying for reasoning power that the work does not require.

We do not need to use one model for an entire session. The `/model` command opens the model picker, allowing us to change models as the task evolves without restarting Codex.

```
Select Model and Effort
Access legacy models by running codex -m <model_name> or in your config.toml

> 1. gpt-5.5 (current) Frontier model for complex coding, research, and real-world work.
   2. gpt-5.4          Strong model for everyday coding.
   3. gpt-5.4-mini     Small, fast, and cost-efficient model for simpler coding tasks.

Press enter to confirm or esc to go back
```

A practical pattern is to begin with the default model, switch to a reasoning-heavy model when we reach a complex problem, and then switch back after resolving it. This gives us extra capability only when it provides real value. We should also choose an appropriate reasoning level for the current task, balancing response speed with the depth of analysis required.

```
Select Reasoning Level for gpt-5.5

1. Low          Fast responses with lighter reasoning
2. Medium (default) Balances speed and reasoning depth for everyday tasks
3. High         Greater reasoning depth for complex problems
4. Extra high   Extra high reasoning depth for complex problems
```

A common mistake is to use the most powerful model for every task. This makes routine work slower and more expensive without necessarily improving the result. Deliberate selection is more effective: start with the default, increase reasoning only when needed, and return to a lighter option once the difficult part is complete.

Use a daily opening ritual

A repeatable opening ritual creates a safe, focused baseline for each Codex session. Before sending the first prompt, follow these five steps:

1. **Commit current changes.** Begin with a clean working directory and create a restore point. If the session produces an unexpected result, we can safely return to the known-good state.

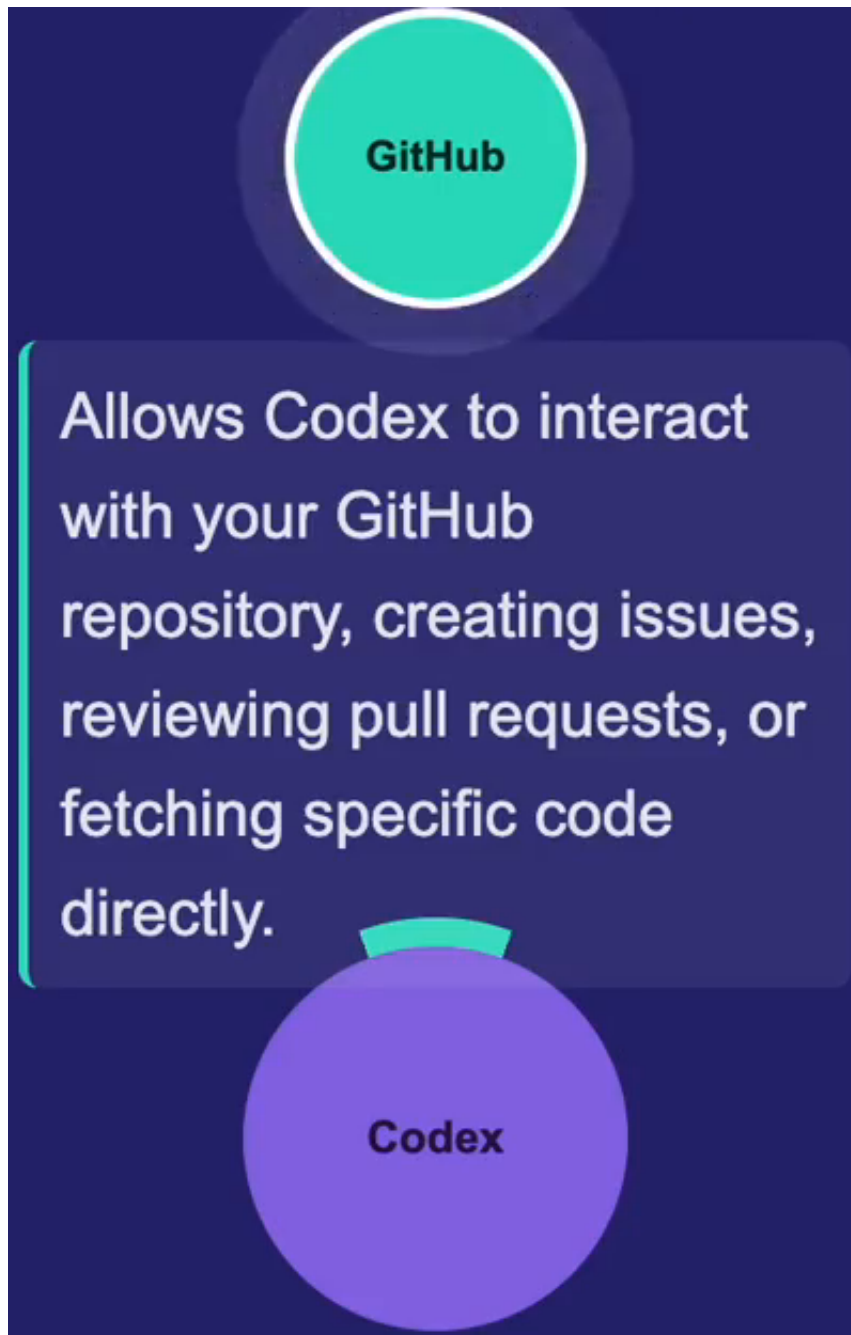


2. **Review AGENTS.md.** Confirm that the project's rules and instructions still reflect its current needs. This quick review helps us identify project drift before it causes larger inconsistencies.
3. **Confirm config.toml.** Check important settings, especially the sandbox and approval policies. This is particularly important before unusual or sensitive work such as a database migration.
4. **Start a fresh session for unrelated work.** Use /new when switching to a different task. A clean context prevents information from the previous task from distracting the agent.
5. **State one concise goal.** Define the session objective in a single sentence before writing the first prompt. If the goal cannot be expressed clearly, the scope is probably too broad and should be narrowed.

This ritual gives Codex a clean baseline, current instructions, suitable permissions, relevant context, and a focused objective. It also reduces the risk of a broad session accumulating unrelated work and producing diluted results.

Extend Codex with MCP servers

MCP servers allow Codex to interact directly with external tools and services. A GitHub MCP server can support repository workflows such as creating issues, reviewing pull requests, or fetching specific code. A database MCP server can expose schemas and support database queries or migrations. Browser and documentation MCP servers can retrieve external information, articles, project knowledge, and current API documentation.



We register an MCP server through `config.toml`, specifying the command that Codex runs to communicate with that external tool. Once registered, the server loads with the session and makes its capabilities available to the workflow.

Know when an MCP server is worthwhile

An MCP server is a strong candidate when we repeatedly transfer information between Codex and another tool during the same session. Copying GitHub issues into prompts or manually fetching database results several times are signs that an integration could remove repetitive work.

We should not add servers speculatively. Every integration creates another potential security surface. Before adding one, verify that we trust its source, understand the access it requires, and expect the workflow benefit to justify that access. The goal is to integrate tools that genuinely improve efficiency, not to increase complexity without a clear need.



Build habits that compound

Model selection, the Git loop, AGENTS.md maintenance, verification practices, and carefully chosen integrations are useful individually. Their greatest value appears when we apply them together and repeat them consistently.

A session that begins with a clean baseline, current rules, correct settings, and a clear scope is more likely to produce a strong result and avoid hours of debugging or rework. We can then turn recurring lessons into permanent improvements: mistakes become project rules, effective procedures become reusable skills, and repeated quality checks become automated safeguards.

The aim is not simply to use Codex effectively once. We are building a workflow that becomes smarter with every interaction. By selecting models deliberately, starting sessions consistently, adding MCP servers only when justified, and preserving what we learn, we create a continuous cycle of safer and more productive development.



Real-World Workflows for AI Development with Codex - Automate and Choose Deliberately

1. You want a final integrity check after Codex finishes each response. Which hook event fits?

- a. Stop
- b. SessionStart
- c. UserPromptSubmit
- d. PreToolUse

2. A difficult multi-file refactor requires deeper architectural reasoning. Which model approach fits best?

- a. Use a reasoning-heavy model for the difficult work
- b. Always use the fastest lightweight model
- c. Change models after every prompt
- d. Choose a model based only on response speed

AI coding tools will continue to change, but the most valuable development skills can transfer between them. By separating durable skills from tool-specific details, we can make learning investments that compound over time instead of resetting whenever a new assistant or interface appears.

Separate the tool from the skill

A tool is a specific implementation used to interact with an AI coding assistant. Codex, Claude Code, Cursor, and Gemini CLI are examples. Each provides its own interface, commands, and workflow.

A skill is the deeper capability behind that interaction. For example, memorizing a command is tool knowledge, while understanding why useful context improves a prompt is a transferable skill. Tool details can change between assistants or versions, but sound problem-solving habits remain useful.

The goal is therefore not only to learn how to use Codex. We can use Codex as a vehicle for building expertise that remains relevant when the tool evolves or is replaced.

Build five durable skills

Five capabilities provide a strong foundation for AI-assisted development:

1. **Plan before building:** Follow a research, plan, and implement loop. Separating thinking from implementation helps prevent impulsive code generation and supports deliberate problem solving.
2. **Scope deliberately:** Break complex features into the smallest independently testable tasks. Small tasks make complexity easier to manage and let us deliver value incrementally.
3. **Verify results:** Give the AI reliable ways to check its work, such as tests, linting, and build scripts. Code that merely looks correct is not enough.
4. **Prompt effectively:** Define the context, task, constraints, and expected output format. Examples, negative constraints, and challenges to the first response can make instructions clearer and results more dependable.
5. **Create a memory system:** Store project conventions and rules in persistent references such as an AGENTS.md file. This avoids repeating the same guidance in every session and helps the AI work consistently.

Expect tool-specific details to change

Some knowledge must be refreshed when we switch tools. This is normal and should not be confused with losing our underlying skills. Common areas of change include:

- Commands and keyboard shortcuts
- Configuration file formats and locations, such as TOML or JSON files
- Permission models and access-control setup
- Hook systems or equivalent automation layers
- Context-window management and monitoring

Learning these details allows us to operate a particular tool, but the durable skills determine how effectively we solve problems with it.

Evaluate a new AI coding tool

When a new tool appears, we can assess its long-term value by asking five practical questions:

1. **Does it have a memory system?** Check whether it can retain persistent project context through a file such as AGENTS.md. Persistent guidance reduces repeated explanations



between sessions.

2. **Can it verify its own output?** Determine whether it can run a test suite, linter, or build script after making changes. Self-verification can reduce debugging time and improve quality.
3. **Does it respect constraints?** Test whether it follows both positive instructions and negative constraints that define what it must not do.
4. **How does it handle context?** Understand its context window, how usage can be monitored, and what happens when the limit is reached. This helps prevent important project information from being lost.
5. **Can it automate repeated workflows?** Look for hooks, custom commands, or similar features that can streamline common paths and reduce manual steps.

This framework keeps attention on capabilities that support reliable development rather than on novelty alone. Tools will continue to evolve, but planning, scoping, verification, clear prompting, and persistent project knowledge provide a stable foundation for lifelong learning.



We have reached the end of the course, and it is time to recognize what we have accomplished. We have done more than learn how to use Codex: we have built a robust, professional workflow around it. Let's recap the core capabilities that make this system effective.

Verification reflex

We have developed the habit of giving Codex a way to check its own work for accuracy. This verification reflex helps minimize errors and improves the quality of the output before we review it ourselves.

Debugging discipline

Our debugging process is now grounded in raw, concrete evidence. By pasting errors directly instead of relying on descriptions or guesswork, we can identify problems and solve them more quickly and accurately.

Subagents

We can delegate research tasks to focused subagents. This protects the main session's context window, allowing Codex to retain the precise information it needs without using that context on exploratory work.

Parallel sessions

We can also run multiple Codex instances at the same time, with each session responsible for a clearly defined part of the project. These ownership boundaries let us work across different areas simultaneously and multiply our output.

Automation stack

Finally, we have built an automation stack using hooks and AGENTS.md. This stack automatically enforces repetitive quality checks, freeing us to focus on complex work that requires human judgment.

Together, verification, evidence-based debugging, subagents, parallel sessions, and automation form a professional Codex workflow that supports accurate and efficient development.

In this lesson, you can find the full source code for the project used within the course. Feel free to use this lesson as needed – whether to help you debug your code, use it as a reference later, or expand the project to practice your skills!

You can also download all project files from the **Course Files** tab via the project files or downloadable PDF summary.

Dev Shop

AGENTS.md

Found in folder: /

This file documents the project's setup, technical stack, development commands, and coding conventions. It establishes strict rules for state management, dependencies, and testing to maintain the project as a controlled learning environment. The rules are designed to keep the codebase simple and focused for students practicing AI-assisted development.

```
# AGENTS.md
```

```
## Project
```

Dev Shop – a React shopping cart starter project for a coding course on AI-assisted development. Students use this as a base to practice adding features with AI assistance.

The app shows a product list on the left and a cart sidebar on the right. All data is hardcoded; there is no backend, API, or localStorage.

```
## Stack
```

- React 19 (functional components only)
- Vite 7
- CSS Modules (no Tailwind, no external UI libraries)
- Vitest + @testing-library/react for tests

```
## Commands
```

Command	Purpose
`npm run dev`	Start dev server (hot reload)
`npm run build`	Production build
`npm run preview`	Preview production build
`npm run test`	Run test suite (Vitest, non-watch)

```
## Conventions
```

- **Functional components only** – no class components
- **CSS Modules only** – one `.module.css` per component, class names in camelCase
- **No external UI libraries** – style from scratch
- **Props down, events up** – all cart state lives in `App.jsx` and is passed to children as props; children emit events via callback props (`onAddToCart`, `onRemove`, `onClearCart`)
- **No routing** – single-page, no React Router
- **Derived values in render** – line totals and cart total are computed inline, not

stored in state

- **Immutable state updates** – always return new arrays from `setCartItems`, never mutate

Critical Rules

1. **Cart state must stay in `App.jsx`.** Do not lift state into a context, store, or separate hook unless explicitly asked. This is a teaching project – the simple prop-drilling pattern is intentional.
2. **Do not add `localStorage`, `sessionStorage`, or any persistence.** State resets on page reload by design.
3. **Do not install additional dependencies** beyond Vite, React, Vitest, and `@testing-library/react`. Students should see a minimal, legible dependency tree.
4. **Missing features are intentional.** This is a starter project. If something seems unimplemented (e.g., quantity controls, checkout flow), assume it is a student exercise – do not silently add it.
5. **All 10 tests must pass** after any change. Run `npm run test` to verify before considering work complete.
6. **ALWAYS run `npm run test` after making any changes to component logic or state management.** Do not consider a task complete until all tests pass.

README.md

Found in folder: /

This file is a README that documents a React shopping cart project. It describes the app's features, tech stack, setup instructions, and project structure. All data is hardcoded with no external dependencies.

Dev Shop – React Shopping Cart

A simple shopping cart app built with React and Vite. Designed as a starter project for a coding course on AI-assisted development.

What it does

- Displays a product catalog on the left
- Lets users add products to a cart sidebar on the right
- Tracks quantities when the same product is added more than once
- Shows per-item line totals and a cart grand total
- Supports removing individual items or clearing the cart entirely

All data is hardcoded – no backend, no API calls, no `localStorage`.

Stack

- **React 19** – functional components, hooks

- **Vite** – dev server and build tool
- **CSS Modules** – scoped per-component styles, no Tailwind or UI libraries
- **Vitest** + **@testing-library/react** – unit/integration tests

Getting started

```
```bash
npm install
npm run dev
```
```

Running tests

```
```bash
npm run test
```
```

The test suite has 10 tests covering product rendering, add-to-cart, quantity tracking, line totals, remove, clear cart, and empty-cart state.

Project structure

```
...
src/
  components/
    ProductList.jsx + ProductList.module.css
    ProductCard.jsx + ProductCard.module.css
    Cart.jsx        + Cart.module.css
    CartItem.jsx    + CartItem.module.css
  data/
    products.js      # hardcoded product catalog
  App.jsx            # root component, owns all cart state
  App.module.css
  main.jsx
  setupTests.js      # @testing-library/jest-dom setup
docs/
  ui.md              # styling conventions
  state.md           # state structure and rules
...

```

Products

```
Name	Price
Mechanical Keyboard	$89.99
USB-C Hub	$34.99
Monitor Stand	$49.99
Webcam HD 1080p	$79.99
Desk Lamp LED	$29.99
...

```

state.md

Found in folder: **/docs**

This document outlines the state management architecture for a shopping cart feature. It specifies that all cart state is centralized in a parent App component, which passes data and callbacks down to child components. The rules define how state is updated immutably and how derived values like totals are calculated.

State Management

Where state lives

****All cart state lives in `App.jsx`.**** No state in child components. No external storage.

```
...
App (owns cartItems state)
??? ProductList ? receives products + onAddToCart prop
?   ??? ProductCard ? receives product + onAddToCart prop
??? Cart ? receives cartItems + onRemove + onClearCart props
    ??? CartItem ? receives item + onRemove props
...
```

State shape

```
```js
cartItems: [
 { id: number, name: string, price: number, quantity: number },
 ...
]
```
```

Each item in `cartItems` is a product object extended with a `quantity` field. Products themselves are never mutated – cart items are copies.

State transitions

| Action | Handler | Result |
|----------------------|----------------------------|--|
| Add new product | `handleAddToCart(product)` | Appends `{ ...product, quantity: 1 }` |
| Add existing product | `handleAddToCart(product)` | Increments `quantity` on matching item |
| Remove item | `handleRemove(productId)` | Filters out item with matching `id` |
| Clear cart | `handleClearCart()` | Resets `cartItems` to `[]` |

Rules

- ****Never mutate state.**** Always return a new array from `setCartItems`.
- ****No localStorage**, no sessionStorage, no backend sync** – state is in-memory only and resets on page reload.
- ****Derived values**** (line totals, cart total) are computed inline in the render – not stored in state.
- ****Props down, events up**** – child components receive data and callbacks; they never reach up to modify parent state directly.

Computed values

| Value | Computed in | Formula |
|------------|-----------------------------|--|
| Line total | <code>`CartItem.jsx`</code> | <code>`item.price * item.quantity`</code> |
| Cart total | <code>`Cart.jsx`</code> | <code>`cartItems.reduce((sum, item) => sum + item.price * item.quantity, 0)`</code> |

ui.md

Found in folder: **/docs**

This markdown file defines the styling conventions and design system for the project. It specifies that CSS Modules are used exclusively for component-scoped styling, with no utility frameworks or external UI libraries. The document outlines layout structure, color palette, typography rules, component styling guidelines, and CSS naming conventions.

UI & Styling Conventions

Approach

All styling uses **CSS Modules** (``.module.css``). No Tailwind, no utility classes, no external UI libraries.

Each component owns its styles in a sibling ``.module.css`` file.

Layout

The app uses a two-column layout defined in ``App.module.css``:

- Left column – ``ProductList`` (fluid, ``flex: 1``)
- Right column – ``Cart`` sidebar (fixed width ``320px``)

The outer wrapper is a flex row with ``gap: 24px`` and ``padding: 32px``.

Color palette

| Role | Value |
|---------------------------|---|
| Page background | <code>`#edf2f7`</code> |
| Card / sidebar background | <code>`#fff`</code> / <code>`#f7fafc`</code> |
| Border | <code>`#e2e8f0`</code> |
| Primary text | <code>`#1a202c`</code> |
| Secondary text | <code>`#4a5568`</code> |
| Muted text | <code>`#718096`</code> |
| Primary action (button) | <code>`#3182ce`</code> hover <code>`#2b6cb0`</code> |
| Destructive (remove) | <code>`#e53e3e`</code> |

Typography

- Base font: system font stack (inherited from browser default)
- Headings use ``font-weight: 700``
- Button labels use ``font-weight: 500``

- No custom font imports

Component style rules

- Cards and sidebars use ``border-radius: 8px`` and ``border: 1px solid #e2e8f0``
- Buttons use ``border-radius: 6px`` (primary) or ``border-radius: 4px`` (small/secondary)
- Transitions on interactive elements: ``0.15s`` duration on ``background`` and ``color``
- No box-shadows – borders only

Naming convention

CSS Module class names use camelCase (e.g., ``.card``, ``.addButton``, ``.lineTotal``). Keep names descriptive and scoped to the component – no global class names.

App.jsx

Found in folder: **/src**

This code defines the main App component for an e-commerce interface. It manages a shopping cart's state, allowing users to add items, remove individual items, and clear the entire cart. It renders a product list and a cart sidebar, passing down the necessary state and handler functions.

```
import { useState } from 'react';
import ProductList from './components/ProductList';
import Cart from './components/Cart';
import products from './data/products';
import styles from './App.module.css';

function App() {
  // State to hold the array of items currently in the shopping cart
  const [cartItems, setCartItems] = useState([]);

  // Adds a product to the cart or increments its quantity if already present
  function handleAddToCart(product) {
    setCartItems((prev) => {
      const existing = prev.find((item) => item.id === product.id);
      if (existing) {
        // Increment quantity for the existing item
        return prev.map((item) =>
          item.id === product.id ? { ...item, quantity: item.quantity + 1 } : item
        );
      }
      // Add new item to cart with initial quantity of 1
      return [...prev, { ...product, quantity: 1 }];
    });
  }

  // Removes an item from the cart based on its product ID
  function handleRemove(productId) {
    setCartItems((prev) => prev.filter((item) => item.id !== productId));
  }
}
```

```
// Clears all items from the cart
function handleClearCart() {
  setCartItems([]);
}

return (
  <div className={styles.layout}>
    <header className={styles.header}>
      <h1 className={styles.title}>Dev Shop</h1>
    </header>
    <main className={styles.main}>
      {/* ProductList displays all available products */}
      <ProductList products={products} onAddToCart={handleAddToCart} />
      {/* Cart displays current cart items and provides removal/clear actions */}
      <Cart
        cartItems={cartItems}
        onRemove={handleRemove}
        onClearCart={handleClearCart}
      />
    </main>
  </div>
);
}

export default App;
```

App.test.jsx

Found in folder: **/src**

This code tests a React shopping cart application. It verifies that product listings display correctly and that cart interactions like adding items, calculating totals, applying discounts, and clearing the cart work as expected. The tests simulate user actions to check the UI updates appropriately.

```
import { render, screen } from '@testing-library/react';
import userEvent from '@testing-library/user-event';
import { describe, it, expect } from 'vitest';
import App from './App';

// Helper function to initialize user events and render the App component
function setup() {
  const user = userEvent.setup();
  render(<App />);
  return { user };
}

describe('Product list', () => {
  it('renders all 5 products', () => {
    setup();
    // Find all "Add to cart" buttons to count products
    const buttons = screen.getAllByRole('button', { name: /add to cart/i });
    expect(buttons).toHaveLength(5);
  });
});
```

```
it('shows each product name and price', () => {
  setup();
  // Assert that all expected product names and prices are displayed
  expect(screen.getByText('Mechanical Keyboard')).toBeInTheDocument();
  expect(screen.getByText('$89.99')).toBeInTheDocument();
  expect(screen.getByText('USB-C Hub')).toBeInTheDocument();
  expect(screen.getByText('$34.99')).toBeInTheDocument();
  expect(screen.getByText('Monitor Stand')).toBeInTheDocument();
  expect(screen.getByText('$49.99')).toBeInTheDocument();
  expect(screen.getByText('Webcam HD 1080p')).toBeInTheDocument();
  expect(screen.getByText('$79.99')).toBeInTheDocument();
  expect(screen.getByText('Desk Lamp LED')).toBeInTheDocument();
  expect(screen.getByText('$29.99')).toBeInTheDocument();
});
});

describe('Add to cart', () => {
  it('adds a product to the cart when "Add to Cart" is clicked', async () => {
    const { user } = setup();
    // Click the first "Add to cart" button
    await user.click(screen.getAllByRole('button', { name: /add to cart/i })[0]);
    // Check that the product name appears in the cart (within a span element)
    expect(screen.getByText('Mechanical Keyboard', { selector: 'span' })).toBeInTheDocument();
  });

  it('increases quantity to 2 when the same product is added twice', async () => {
    const { user } = setup();
    const addButton = screen.getAllByRole('button', { name: /add to cart/i })[0];
    await user.click(addButton);
    await user.click(addButton);
    // Expect to see a quantity indicator like "× 2"
    expect(screen.getByText(/× 2/)).toBeInTheDocument();
  });

  it('shows correct line total for a product with quantity 2', async () => {
    const { user } = setup();
    const addButton = screen.getAllByRole('button', { name: /add to cart/i })[0];
    await user.click(addButton);
    await user.click(addButton);
    // Mechanical Keyboard $89.99 × 2 = $179.98 (line total and original total are both $179.98)
    expect(screen.getAllByText('$179.98')).toHaveLength(2);
  });

  it('shows correct total when multiple different products are added', async () => {
    const { user } = setup();
    const addButtons = screen.getAllByRole('button', { name: /add to cart/i });
    // Mechanical Keyboard $89.99 + USB-C Hub $34.99 = $124.98
    await user.click(addButtons[0]);
    await user.click(addButtons[1]);
    expect(screen.getByText(/\$124\.\98/)).toBeInTheDocument();
  });
});
```



```
it('does not show a discount when the cart total is $100 or less', async () => {
  const { user } = setup();
  await user.click(screen.getAllByRole('button', { name: /add to cart/i })[0]);

  expect(screen.getByText('Original total:')).toBeInTheDocument();
  expect(screen.getByText('Total:')).toBeInTheDocument();
  // Discount section should not appear for totals ? $100
  expect(screen.queryByText('Discount:')).not.toBeInTheDocument();
});

it('applies a 10% discount when the cart total exceeds $100', async () => {
  const { user } = setup();
  const addButtons = screen.getAllByRole('button', { name: /add to cart/i });
  await user.click(addButtons[0]);
  await user.click(addButtons[1]);

  expect(screen.getByText('Original total:')).toBeInTheDocument();
  expect(screen.getByText('$124.98')).toBeInTheDocument();
  expect(screen.getByText('Discount:')).toBeInTheDocument();
  expect(screen.getByText('- $12.50')).toBeInTheDocument();
  expect(screen.getByText('$112.48')).toBeInTheDocument();
});

describe('Cart interactions', () => {
  it('removes an item entirely when "Remove" is clicked', async () => {
    const { user } = setup();
    await user.click(screen.getAllByRole('button', { name: /add to cart/i })[0]);
    // Click the "Remove" button for the added item
    await user.click(screen.getByRole('button', { name: /remove/i }));
    // Verify the product is no longer in the cart
    expect(screen.queryByText('Mechanical Keyboard', { selector: 'span' })).not.toBeInTheDocument();
  });

  it('empties the cart when "Clear Cart" is clicked', async () => {
    const { user } = setup();
    const addButtons = screen.getAllByRole('button', { name: /add to cart/i });
    await user.click(addButtons[0]);
    await user.click(addButtons[1]);
    await user.click(screen.getByRole('button', { name: /clear cart/i }));
    // Check that the empty cart message appears
    expect(screen.getByText(/your cart is empty/i)).toBeInTheDocument();
  });
});

describe('Empty cart state', () => {
  it('shows "Your cart is empty" when no items are in the cart', () => {
    setup();
    expect(screen.getByText(/your cart is empty/i)).toBeInTheDocument();
  });

  it('does not show "Your cart is empty" when the cart has items', async () => {
    const { user } = setup();
    await user.click(screen.getAllByRole('button', { name: /add to cart/i })[0]);
```

```
    // The empty message should disappear after adding an item
    expect(screen.queryByText(/your cart is empty/i)).not.toBeInTheDocument();
  });
});
```

main.jsx

Found in folder: **/src**

This code initializes a React application by rendering the main App component inside a React StrictMode wrapper into the HTML element with the id 'root'. It uses React 18's createRoot API for concurrent rendering features. The StrictMode enables additional development checks for potential problems.

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

// Initialize React root on the DOM element with id 'root'
createRoot(document.getElementById('root')).render(
  <StrictMode>
    /* Main application component wrapped in StrictMode for development checks */
    <App />
  </StrictMode>
);
```

Cart.jsx

Found in folder: **/src/components**

This component renders a shopping cart sidebar. It displays a list of cart items, calculates a total and a 10% discount if the total exceeds \$100, and shows a summary. It also provides a button to clear all items from the cart.

```
import CartItem from './CartItem';
import styles from './Cart.module.css';

function Cart({ cartItems, onRemove, onClearCart }) {
  // Calculate the total price of all items in the cart
  const total = cartItems.reduce((sum, item) => sum + item.price * item.quantity, 0);
  // Apply a 10% discount if the total exceeds $100
  const discountAmount = total > 100 ? total * 0.1 : 0;
  const discountedTotal = total - discountAmount;

  return (
    <aside className={styles.sidebar}>
      <h2 className={styles.heading}>Cart</h2>

      /* Conditionally render empty message or cart contents */
      {cartItems.length === 0 ? (
        <p className={styles.empty}>Your cart is empty</p>
      ) : (
        <div>
          {cartItems.map(item => <CartItem key={item.id} item={item} onRemove={onRemove} />)}
          <div>
            Total: ${discountedTotal.toFixed(2)}
            <button className={styles.clear} onClick={onClearCart}>Clear Cart</button>
          </div>
        </div>
      )}
    </aside>
  );
}
```

```
    ) : (
      <>
        <div className={styles.items}>
          {/* Render each cart item as a separate CartItem component */}
          {cartItems.map((item) => (
            <CartItem key={item.id} item={item} onRemove={onRemove} />
          ))}
        </div>
        <div className={styles.footer}>
          <div className={styles.summary}>
            <p className={styles.summaryRow}>
              Original total: <strong>${total.toFixed(2)}</strong>
            </p>
            {/* Only show discount line if a discount was applied */}
            {discountAmount > 0 && (
              <p className={styles.summaryRow}>
                Discount: <strong>-${discountAmount.toFixed(2)}</strong>
              </p>
            )}
            <p className={styles.total}>
              Total: <strong>${discountedTotal.toFixed(2)}</strong>
            </p>
          </div>
          <button className={styles.clear} onClick={onClearCart}>
            Clear Cart
          </button>
        </div>
      </>
    )}
  </aside>
);
}

export default Cart;
```

CartItem.jsx

Found in folder: **/src/components**

This component renders a single item in a shopping cart, displaying its name, price, quantity, and calculated total. It provides a button to remove the item from the cart by calling the provided `onRemove` function with the item's ID.

```
import styles from './CartItem.module.css';

function CartItem({ item, onRemove }) {
  // Calculate the total cost for this line item
  const lineTotal = item.price * item.quantity;

  return (
    <div className={styles.item}>
      <div className={styles.details}>
        <span className={styles.name}>{item.name}</span>
```

```
        <span className={styles.meta}>
          ${item.price.toFixed(2)} × {item.quantity} ={' '}
          <strong>${lineTotal.toFixed(2)}</strong>
        </span>
      </div>
      { /* Calls onRemove with the item's id when clicked */ }
      <button className={styles.remove} onClick={() => onRemove(item.id)}>
        Remove
      </button>
    </div>
  );
}

export default CartItem;
```

ProductCard.jsx

Found in folder: **/src/components**

This code defines a React component that displays a product's name and price in a styled card, and includes a button that triggers a function to add that product to a cart when clicked.

```
import styles from './ProductCard.module.css';

// Component to display a single product's information and an action button.
function ProductCard({ product, onAddToCart }) {
  return (
    <div className={styles.card}>
      <div className={styles.info}>
        <h4 className={styles.name}>{product.name}</h4>
        <p className={styles.price}>${product.price.toFixed(2)}</p> { /* Formats price
to two decimal places */ }
      </div>
      <button className={styles.button} onClick={() => onAddToCart(product)}>
        Add to Cart
      </button>
    </div>
  );
}

export default ProductCard;
```

ProductList.jsx

Found in folder: **/src/components**

This code renders a list of products by mapping over a provided array. It displays each product using a `ProductCard` component, passing along product data and an add-to-cart handler. The layout is styled using CSS modules.

```
import ProductCard from './ProductCard';
```

```
import styles from './ProductList.module.css';

// Renders a grid of product cards
function ProductList({ products, onAddToCart }) {
  return (
    <section className={styles.list}>
      <h2 className={styles.heading}>Products</h2>
      <div className={styles.grid}>
        {/* Iterate over products array to create a card for each */}
        {products.map((product) => (
          <ProductCard
            key={product.id} // Unique key for React list rendering
            product={product}
            onAddToCart={onAddToCart}
          />
        ))}
      </div>
    </section>
  );
}

export default ProductList;
```

index.html

Found in folder: /

This HTML file sets up the basic structure of a web page, including metadata and viewport settings for responsive design. It links to a favicon and loads a React application by mounting it into a `div` element with the id "root". The page's JavaScript entry point is a module script located at `/src/main.jsx`.

```
<!doctype html>
<html lang="fr">
  <head>
    <meta charset="UTF-8" />
    <link rel="icon" type="image/svg+xml" href="/vite.svg" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Vite + React</title>
  </head>
  <body>
    <!-- Root container where the React app will be rendered -->
    <div id="root"></div>
    <!-- Entry point for the application's JavaScript, using ES modules -->
    <script type="module" src="/src/main.jsx"></script>
  </body>
</html>
```

App.module.css

Found in folder: /src



This CSS defines a responsive page layout with a dark header and a light flex container. It uses a column flexbox for the main content area and adjusts padding and orientation on smaller screens. The media query changes the layout to a vertical stack on mobile devices.

```
.layout {
  min-height: 100vh;          /* Full viewport height minimum */
  background: #edf2f7;        /* Light gray background */
  display: flex;
  flex-direction: column;     /* Stack children vertically */
}

.header {
  background: #1a202c;        /* Dark header background */
  padding: 16px 32px;
}

.title {
  margin: 0;
  font-size: 1.5rem;
  font-weight: 700;
  color: #fff;                /* White text */
  letter-spacing: 0.025em;
}

.main {
  display: flex;
  align-items: flex-start;    /* Align items to the top */
  gap: 24px;                  /* Space between flex children */
  padding: 32px;
  flex: 1;                   /* Take remaining vertical space */
}

@media (max-width: 640px) {
  .header {
    padding: 12px 16px;      /* Reduced padding on mobile */
  }

  .main {
    flex-direction: column;   /* Stack items vertically on small screens */
    padding: 16px;
    gap: 16px;
  }
}
```

Cart.module.css

Found in folder: **/src/components**

This CSS styles a sidebar component with a clean, responsive design. It defines visual styles for elements like headings, items, summaries, and a clear button, including hover effects. On smaller screens, the sidebar adjusts to full width and changes its alignment.

```
.sidebar {
```



```
width: 320px;
flex-shrink: 0; /* Prevents sidebar from shrinking in a flex container */
background: #f7fafc;
border: 1px solid #e2e8f0;
border-radius: 8px;
padding: 20px;
display: flex;
flex-direction: column;
align-self: flex-start; /* Aligns to the top of a flex container */
}

@media (max-width: 640px) {
  .sidebar {
    width: 100%; /* Makes sidebar responsive on small screens */
    align-self: stretch; /* Fills available width */
  }
}

.heading {
  margin: 0 0 16px;
  font-size: 1.5rem;
  font-weight: 700;
  color: #1a202c;
}

.empty {
  color: #718096;
  font-size: 0.9rem;
  margin: 0;
}

.items {
  display: flex;
  flex-direction: column;
}

.footer {
  margin-top: 16px;
  display: flex;
  flex-direction: column;
  gap: 12px; /* Spacing between child elements */
}

.summary {
  display: flex;
  flex-direction: column;
  gap: 6px;
}

.summaryRow,
.total {
  margin: 0;
  font-size: 1rem;
  color: #1a202c;
}
```

```
.summaryRow {
  display: flex;
  justify-content: space-between;
  gap: 12px;
}

.summaryRow:nth-child(2) {
  color: #2f855a; /* Highlights a specific summary row, e.g., for discounts */
}

.total {
  display: flex;
  justify-content: space-between;
  gap: 12px;
  padding-top: 8px;
  border-top: 1px solid #e2e8f0; /* Visual separator for total row */
}

.clear {
  padding: 8px 16px;
  background: transparent;
  color: #718096;
  border: 1px solid #cbd5e0;
  border-radius: 6px;
  font-size: 0.875rem;
  cursor: pointer;
  transition: background 0.15s, border-color 0.15s; /* Smooth hover transition */
}

.clear:hover {
  background: #edf2f7;
  border-color: #a0aec0;
}
```

CartItem.module.css

Found in folder: **/src/components**

This CSS styles a list item component with a flexible layout, including details and a remove button. It defines visual styles for text elements and an interactive button with hover effects. The last item's bottom border is removed for cleaner spacing.

```
.item {
  display: flex;
  align-items: flex-start;
  justify-content: space-between;
  padding: 12px 0;
  border-bottom: 1px solid #e2e8f0;
  gap: 8px;
}

.item:last-of-type {
```



```
border-bottom: none; /* Removes border from the last item */
}

.details {
  display: flex;
  flex-direction: column;
  gap: 4px;
  flex: 1; /* Allows details to take available space */
  min-width: 0; /* Prevents overflow in flex container */
}

.name {
  font-size: 0.9rem;
  font-weight: 600;
  color: #1a202c;
}

.meta {
  font-size: 0.8rem;
  color: #4a5568;
}

.remove {
  padding: 4px 10px;
  background: transparent;
  color: #e53e3e;
  border: 1px solid #e53e3e;
  border-radius: 4px;
  font-size: 0.75rem;
  cursor: pointer;
  white-space: nowrap; /* Prevents button text from wrapping */
  transition: background 0.15s, color 0.15s;
}

.remove:hover {
  background: #e53e3e;
  color: #fff;
}
```

ProductCard.module.css

Found in folder: **/src/components**

This CSS defines styles for a product card component, including layout, typography, and interactive button states. It creates a flex container for the card with spaced alignment and a hover effect for the button. The styles set visual properties like colors, borders, and spacing for the card's elements.

```
.card {
  display: flex;
  align-items: center;
  justify-content: space-between;
  padding: 16px;
  background: #fff;
}
```



```
border: 1px solid #e2e8f0;
border-radius: 8px;
gap: 12px; /* Space between child elements */
}

.info {
  display: flex;
  flex-direction: column;
  gap: 4px; /* Vertical spacing between name and price */
}

.name {
  margin: 0;
  font-size: 1rem;
  font-weight: 600;
  color: #1a202c;
}

.price {
  margin: 0;
  font-size: 0.9rem;
  color: #4a5568;
}

.button {
  padding: 8px 16px;
  background: #3182ce;
  color: #fff;
  border: none;
  border-radius: 6px;
  font-size: 0.875rem;
  font-weight: 500;
  cursor: pointer;
  white-space: nowrap; /* Prevents button text from wrapping */
  transition: background 0.15s; /* Smooth color change on hover */
}

.button:hover {
  background: #2b6cb0; /* Darker blue on hover */
}
```

ProductList.module.css

Found in folder: **/src/components**

This CSS defines three classes: `.list`` sets flexible sizing with a minimum width, `.heading`` styles a bold, large, dark header with bottom margin, and `.grid`` creates a vertical flex container with spacing between items.

```
.list {
  flex: 1;          /* Allows element to grow/shrink, taking available space */
  min-width: 0;     /* Prevents flex item from overflowing its container */
}
```

```
.heading {
  margin: 0 0 16px; /* Top:0, Right:0, Bottom:16px, Left:0 */
  font-size: 1.5rem;
  font-weight: 700;
  color: #1a202c; /* Dark gray color */
}

.grid {
  display: flex;
  flex-direction: column; /* Stack children vertically */
  gap: 12px; /* Space between child elements */
}
```

products.js

Found in folder: **/src/data**

This code defines an array of product objects with properties like id, name, and price, then exports it as the default export for use in other modules.

```
const products = [
  { id: 1, name: 'Mechanical Keyboard', price: 89.99 },
  { id: 2, name: 'USB-C Hub', price: 34.99 },
  { id: 3, name: 'Monitor Stand', price: 49.99 },
  { id: 4, name: 'Webcam HD 1080p', price: 79.99 },
  { id: 5, name: 'Desk Lamp LED', price: 29.99 },
];

// Export the products array as the default module export
export default products;
```

Finance Tracker

AGENTS.md

Found in folder: /

This document outlines the project structure, technical stack, and development rules for a personal finance tracker. It specifies the commands, file naming conventions, and strict data flow where all business logic is isolated to a service layer. The rules enforce a static, hardcoded implementation without external dependencies or state persistence.

Finance Tracker – AGENTS.md

Project

A personal finance tracker built as a starter project for a coding course on AI-assisted development. Displays a list of transactions, a summary dashboard with totals by category, and a monthly balance view. All data is hardcoded – no backend, no API, no localStorage.

Stack

```
| Layer | Technology |
|---|---|
| Framework | React 19 + Vite 7 |
| Styling | CSS Modules (plain CSS, no Tailwind) |
| Testing | Vitest 4 + @testing-library/react |
| Linting | ESLint 9 (flat config) |
| Language | JavaScript (ESM) |
```

Commands

```
```bash
npm run dev # start dev server at http://localhost:5173
npm run build # production build ? dist/
npm run preview # preview the production build
npm run lint # lint src/ with ESLint
npm run test # run Vitest in watch mode
npm run test:run # run tests once (CI mode)
```
```

Conventions

File structure

```
```
src/
 components/ # React components, one per file + matching .module.css
 services/ # All business logic and data access
 utils/ # Pure utility functions (formatting, colour mapping)
 data/ # Hardcoded mock data
 App.jsx # Root component – wires service layer to components
 main.jsx # React entry point
```
```

Naming

- Components: PascalCase (`TransactionItem.jsx`)
- CSS modules: same name as component (`TransactionItem.module.css`)
- Services and utils: camelCase (`transactionService.js`, `currencyUtils.js`)
- Test files: co-located with the module they test, `.test.js` suffix

CSS

- Plain CSS modules only – no Tailwind, no external UI libraries
- Colour palette: slate grays for text (`#0f172a`, `#1e293b`, `#64748b`), green `#16a34a` for income, red `#dc2626` for expenses
- Component styles are scoped; global resets live in `App.module.css` only

Data flow

- Service functions in `transactionService.js` are the single source of truth
- `App.jsx` calls the service layer once at module load (not inside render)
- Components receive data via props – they never import from services or data files directly

Critical Rules

1. ****All calculations must happen in `transactionService.js`**** – components only receive and display data. Never put filtering, summing, or aggregation logic inside a component.
2. ****No new dependencies**** – the allowed set is: React, ReactDOM, Vite, Vitest, @testing-library. Do not add UI libraries, date libraries, or state managers.
3. ****No localStorage, no fetch, no backend**** – this is a fully static, hardcoded project.
4. ****12 tests must stay green**** – do not modify the test files without keeping all 12 passing.
5. ****`npm run lint` must pass with zero errors**** – fix lint errors before committing.
6. ****The project contains intentional bugs**** seeded for student exercises – do not silently fix code that looks wrong without reading the course context first.

README.md

Found in folder: /

This Markdown file serves as the project documentation for a personal finance tracker application. It outlines the application's features, setup instructions, project structure, and technical architecture. The document is intended to guide developers on how to run, build, and understand the codebase.

Finance Tracker

A React + Vite personal finance tracker built as a starter project for a coding course on AI-assisted development.

What it does

- Displays 12 hardcoded transactions for January 2025
- Shows a summary dashboard with total income, total expenses, and current balance
- Breaks down spending by category with colour-coded labels
- Highlights whether the month is in profit or loss

Getting started

```
```bash
npm install
npm run dev
```
```

Open <http://localhost:5173>.

Scripts

| Command | Description |
|-------------------|------------------------------|
| `npm run dev` | Start the development server |
| `npm run build` | Build for production |
| `npm run preview` | Preview the production build |
| `npm run lint` | Lint `src/` with ESLint |
| `npm run test` | Run the Vitest test suite |

Project structure

...

```
src/
  components/      React UI components (CSS Modules)
  services/        Business logic – all calculations live here
  utils/           Pure utility functions (currency formatting, colours)
  data/            Hardcoded transaction data
  App.jsx          Root component
  main.jsx         Entry point
docs/
  ui.md            Styling conventions
  state.md         Data flow and services layer rules
  testing.md       Testing conventions
AGENTS.md         Guide for Codex AI assistant
...
```

Tech stack

- **React 19** – UI
- **Vite 7** – build tool and dev server
- **CSS Modules** – scoped component styles, no external UI library
- **Vitest** – unit tests (12 tests covering services and utils)
- **ESLint 9** – flat config with React Hooks plugin

Design decisions

All data flows top-down: ``data/transactions.js`` ? ``services/transactionService.js`` ? ``App.jsx`` ? components via props. Components contain no business logic. This separation makes the service layer easy to test in isolation and straightforward to swap for a real API later.

state.md

Found in folder: **/docs**

This document outlines the data flow architecture and rules for a static financial application. It describes a unidirectional data flow from a hardcoded source, through a service layer for calculations, down to components via props. It establishes strict rules to keep business logic in services and display logic in components.

Data Flow & State

Architecture overview

...

```
src/data/transactions.js      ? single source of truth (hardcoded array)
  ?
src/services/transactionService.js ? all calculations and queries
  ?
src/App.jsx                   ? calls service functions once at module load
  ?
components (via props)        ? display only, no logic
```

```
...
```

There is no state management library. There is no `useState` or `useReducer` in this project – data does not change at runtime, so all values are computed once and passed as props.

Services layer rules

All business logic lives in `src/services/transactionService.js`. The exported functions are:

```
Function	Returns
`getAllTransactions()`	Full array of 12 transactions
`getTransactionsByCategory(category)`	Filtered array
`getTotalIncome()`	Number (positive)
`getTotalExpenses()`	Number (negative)
`getBalance()`	Number (income + expenses)
`getSummaryByCategory()`	`Array<{ category: string, total: number }>`
```

Rules:

- Components must never import from `src/data/transactions.js` directly.
- Components must never perform filtering, summing, or aggregation.
- If you need a new derived value, add a function to `transactionService.js` and call it from `App.jsx`.

Props contract

`App.jsx` calls the service layer once and passes results down:

```
...
```

```
App
??? TransactionList ({ transactions })
?   ??? TransactionItem ({ transaction }) ? per item
??? Dashboard ({ income, expenses, balance, summary })
    ??? MonthlyBalance ({ balance, month })
    ??? CategorySummary ({ summary })
...
```

No component reaches up for data – data flows strictly downward via props.

Utils

Utils are pure functions with no side effects:

- `src/utils/currencyUtils.js` – `formatCurrency(amount)` ? formatted string
- `src/utils/categoryUtils.js` – `getCategoryColour(category)` ? hex colour string

Components may import utils directly since they are stateless helpers, not data sources.

File extension: md

testing.md

Found in folder: **/docs**

This markdown file documents the testing conventions for the project, including the framework used, file structure, import style, and a specific list of required tests with their expected outcomes.

Testing Conventions

Framework

[Vitest](https://vitest.dev/) – configured in `vitest.config.js` with the jsdom environment.

Run tests:

```
```bash
npm run test # watch mode
vitest --run # single pass (CI)
```
```

Test file location

Test files live alongside the module they test, using the `.test.js` suffix:

```
...
src/services/transactionService.js
src/services/transactionService.test.js

src/utils/currencyUtils.js
src/utils/currencyUtils.test.js

src/utils/categoryUtils.js
src/utils/categoryUtils.test.js
...
```

Import style

Always import Vitest globals explicitly – `globals: false` is set in `vitest.config.js`:

```
```js
import { describe, it, expect } from 'vitest';
```
```

What must be tested

The 12 required tests and what they verify:

| # | Test | Expected value |
|---|-------------------------------------|-------------------|
| 1 | `getAllTransactions` | length 12 |
| 2 | `getTotalIncome` | `4000` |
| 3 | `getTotalExpenses` | `-1493` |
| 4 | `getBalance` | `2507` |
| 5 | `getTransactionsByCategory('Food')` | 3 items, all Food |



```
6	`getTransactionsByCategory('Unknown')`	`[]`
7	`getSummaryByCategory` length	8
8	`getSummaryByCategory` Food total	`-230`
9	`formatCurrency(3200)`	`£3,200.00`
10	`formatCurrency(-950)`	`'-£950.00'`
11	`getCategoryColour` for all 8 known categories	returns a string
12	`getCategoryColour('Unknown')`	`'#6b7280'` (fallback)
```

Rules

- All 12 tests must pass before any PR is merged.
- Do not mock the data layer in service tests – tests should use the real `transactions.js` data so they catch regressions in both the data and the service functions.
- Do not test component rendering unless a new component test is specifically required – the current suite focuses on the service and util layers.
- If you add a new service function, add at least one test for it.

ui.md

Found in folder: **/docs**

This markdown file documents the UI and styling conventions for the project, including CSS module usage, layout structure, color palette, and typography rules. It establishes consistent design patterns for components like cards and color-coded amounts. The guidelines ensure all styling is implemented with plain CSS according to the specified standards.

UI & Styling Conventions

CSS Modules

Every component has its own `.module.css` file with the same base name. Import styles as a default import named `styles`:

```
```js
import styles from './ComponentName.module.css';
// use as: className={styles.container}
```
```

No Tailwind, no styled-components, no external UI library. All styling is plain CSS.

Layout

The root layout is a two-column CSS grid defined in `App.module.css`:

```
```css
.layout {
 display: grid;
 grid-template-columns: 1fr 380px;
 gap: 24px;
 padding: 28px 32px;
}
```

- Left column: `TransactionList` (flexible width)
- Right column: `Dashboard` (fixed 380px)
- Collapses to single column below 768px

### ## Colour palette

Role	Value
Page background	`#f1f5f9`
Card background	`#ffffff`
Primary text	`#0f172a`
Secondary text	`#1e293b`
Muted text	`#64748b`
Subtle text / labels	`#94a3b8`
Border / divider	`#f1f5f9`, `#e2e8f0`
Income (green)	`#16a34a`
Expense (red)	`#dc2626`
Profit background	`#f0fdf4`
Loss background	`#fef2f2`

Category colours are defined in `src/utils/categoryUtils.js` and returned by `getCategoryColour(category)`. Never hardcode category colours in component files.

### ## Card style

Panels (TransactionList, Dashboard) use:

```
```css
background: #ffffff;
border-radius: 12px;
padding: 24px;
box-shadow: 0 1px 3px rgba(0, 0, 0, 0.08);
```
```

### ## Typography

- Font stack: `-apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif`
- Section headings: `1.1rem`, `font-weight: 700`
- Category/stat labels: `0.78-0.82rem`, uppercase, `letter-spacing: 0.05em`
- Transaction description: `0.95rem`, `font-weight: 500`
- Amount values: `0.95rem`, `font-weight: 600`

### ## Colour-coding rule

- Any amount that is positive (income) ? `#16a34a` green
- Any amount that is negative (expense) ? `#dc2626` red
- Monthly balance card ? green background/text if balance ? 0, red if < 0

File extension: md

## MEMORY.md

Found in folder: **/memory**

This code provides a memory index linking to a project starter file for a personal finance tracker course. It notes the starter includes a full built project with 12 passing tests and instructs not to silently fix seeded bugs.

# Memory Index

- [Finance Tracker Course Starter](project\_todo\_course\_starter.md) – personal finance tracker starter; full project built, 12 tests passing; do not silently fix seeded bugs

## App.jsx

Found in folder: **/src**

This code defines the main App component that renders a finance tracker interface. It imports transaction data and calculations from a service, then passes them to child components for display. The component structures the layout with a header and two main sections: a transaction list and a dashboard.

```
import styles from './App.module.css';
import TransactionList from './components/TransactionList.jsx';
import Dashboard from './components/Dashboard.jsx';
import {
 getAllTransactions,
 getTotalIncome,
 getTotalExpenses,
 getBalance,
 getExpenseSummaryByCategory,
} from './services/transactionService.js';

// Load transaction data and calculations once when module loads
const transactions = getAllTransactions();
const income = getTotalIncome();
const expenses = getTotalExpenses();
const balance = getBalance();
const summary = getExpenseSummaryByCategory();

function App() {
 return (
 <div className={styles.app}>
 <header className={styles.header}>
 <h1 className={styles.title}>Finance Tracker</h1>
 <p className={styles.subtitle}>January 2026</p>
 </header>
 <main className={styles.layout}>
 </* Pass transaction data to list component */>
 <TransactionList transactions={transactions} />
 </* Pass calculated financial data to dashboard component */>
 <Dashboard
 income={income}
 expenses={expenses}
 balance={balance}
 summary={summary}
 />
 </main>
 </div>
);
}
```

```
 />
 </main>
 </div>
);
}

export default App;
```

## main.jsx

Found in folder: **/src**

This code initializes a React application by rendering the main App component into the DOM element with the ID 'root'. It uses React's StrictMode to highlight potential problems during development. The App component is imported from a local file and wrapped in StrictMode for additional checks.

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

// Create a root for React to render into the DOM element with ID 'root'
const root = createRoot(document.getElementById('root'));

// Render the App component wrapped in StrictMode for development checks
root.render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

## CategorySummary.jsx

Found in folder: **/src/components**

This component displays a list of spending categories, their totals, and a visual warning for categories that have exceeded their budget. It uses a colored dot for each category and formats the currency values. An exclamation mark with a tooltip is shown next to categories that are over budget.

```
import styles from './CategorySummary.module.css';
import { formatCurrency } from '../utils/currencyUtils.js';
import { getCategoryColour } from '../utils/categoryUtils.js';

function CategorySummary({ summary }) {
 return (
 <div className={styles.container}>
 <h4 className={styles.heading}>By Category</h4>
 <ul className={styles.list}>
 {/* Render a list item for each category in the summary prop */}
 {summary.map((item) => (
 <li key={item.category} className={styles.row}>
```



```
 {/* Colored dot representing the category */}
 <span
 className={styles.dot}
 style={{ backgroundColor: getCategoryColour(item.category) }}
 />
 {item.category}
 {formatCurrency(item.total)}
 {/* Show warning icon if this category is over budget */}
 {item.isOverBudget && (
 <span
 className={styles.warning}
 aria-label=
label={`$${item.category} is over budget by ${formatCurrency(item.overBudgetBy)}`}
 title={`$${item.category} is over budget by ${formatCurrency(item.over
BudgetBy)}`}
 >
 !

)}

)}

</div>
);
}
```

export default CategorySummary;

## Dashboard.jsx

Found in folder: **/src/components**

This code renders a dashboard sidebar component displaying financial overview data. It shows a monthly balance, total income and expenses in styled cards, and a category breakdown. The component uses imported helper functions and child components to format and present the data.

```
import styles from './Dashboard.module.css';
import CategorySummary from './CategorySummary.jsx';
import MonthlyBalance from './MonthlyBalance.jsx';
import { formatCurrency } from '../utils/currencyUtils.js';

function Dashboard({ income, expenses, balance, summary }) {
 return (
 <aside className={styles.container}>
 <h2 className={styles.heading}>Dashboard</h2>

 {/* Displays the balance for a specific month */}
 <MonthlyBalance balance={balance} month="January" />

 <div className={styles.stats}>
 {/* Card for total income */}
 <div className={styles.statCard}>
 <p className={styles.statLabel}>Total Income</p>

```



```
 <p className={` ${styles.statValue} ${styles.income}`}>{formatCurrency(income)}</p>
 </div>
 { /* Card for total expenses */ }
 <div className={styles.statCard}>
 <p className={styles.statLabel}>Total Expenses</p>
 <p className={` ${styles.statValue} ${styles.expense}`}>{formatCurrency(expenses)}</p>
 </div>
 </div>

 { /* Shows a breakdown of expenses or income by category */ }
 <CategorySummary summary={summary} />
 </aside>
);
}

export default Dashboard;
```

## MonthlyBalance.jsx

Found in folder: **/src/components**

This component displays a monthly balance with conditional styling based on whether it's positive or negative. It shows the month name, formatted balance amount, and a profit/loss status label. The visual appearance changes between profit and loss states using different CSS classes.

```
import styles from './MonthlyBalance.module.css';
import { formatCurrency } from '../utils/currencyUtils.js';

function MonthlyBalance({ balance, month }) {
 const isProfit = balance >= 0; // Determine if balance is positive (profit) or negative (loss)

 return (
 // Apply different styling based on profit/loss status
 <div className={` ${styles.container} ${isProfit ? styles.profit : styles.loss}`}>
 <p className={styles.label}>{month} Balance</p>
 <p className={styles.amount}>{formatCurrency(balance)}</p>
 <p className={styles.status}>{isProfit ? 'In profit' : 'In loss'}</p>
 </div>
);
}

export default MonthlyBalance;
```

## TransactionItem.jsx

Found in folder: **/src/components**

This component renders a single transaction as a list item, displaying its description, category, and

formatted amount. It uses a colored dot to visually represent the transaction's category and styles the amount differently based on whether it is income or an expense.

```
import styles from './TransactionItem.module.css';
import { formatCurrency } from '../utils/currencyUtils.js';
import { getCategoryColour } from '../utils/categoryUtils.js';

function TransactionItem({ transaction }) {
 // Destructure transaction properties for easier access
 const { description, amount, category } = transaction;
 const isIncome = amount > 0; // Determine if transaction is income (positive amount)
}

return (
 <li className={styles.item}>
 /* Coloured dot representing the transaction category */
 <span
 className={styles.dot}
 style={{ backgroundColor: getCategoryColour(category) }}
 />
 <div className={styles.details}>
 {description}
 {category}
 </div>
 /* Amount styled differently for income vs expense */
 <span className={` ${styles.amount} ${isIncome ? styles.income : styles.expense}`
 >{formatCurrency(amount)}

);
}

export default TransactionItem;
```

## TransactionList.jsx

Found in folder: **/src/components**

This component displays a list of transactions and provides a search input to filter them by description. It uses a state variable to track the search text and filters the transactions based on whether their description includes that text. The filtered list is then rendered using a separate `TransactionItem` component for each transaction.

```
import { useState } from 'react';
import styles from './TransactionList.module.css';
import TransactionItem from './TransactionItem.jsx';

function TransactionList({ transactions }) {
 // State for the user's search input
 const [searchText, setSearchText] = useState('');
 // Normalize search text by trimming whitespace and converting to lowercase for consistent comparison
```

```
const normalizedSearchText = searchText.trim().toLowerCase();
// Filter transactions based on search text; show all if search is empty
const visibleTransactions = normalizedSearchText
 ? transactions.filter((transaction) =>
 transaction.description.toLowerCase().includes(normalizedSearchText),
)
 : transactions;

return (
 <section className={styles.container}>
 <h2 className={styles.heading}>Transactions</h2>
 <label className={styles.searchLabel} htmlFor="transaction-search">
 Search transactions
 </label>
 <input
 className={styles.searchInput}
 id="transaction-search"
 type="search"
 value={searchText}
 onChange={(event) => setSearchText(event.target.value)}
 placeholder="Search by description"
 />
 <ul className={styles.list}>
 {visibleTransactions.map((transaction) => (
 <TransactionItem key={transaction.id} transaction={transaction} />
))}

 </section>
);
}

export default TransactionList;
```

## index.html

Found in folder: /

This HTML file defines the basic structure of a web page, setting up metadata, linking resources, and creating a container where a React application will be mounted. It specifies the document type, language, character set, viewport for responsive design, and the page title. The `

` is the entry point for the React app, which is loaded by the script module `src/main.jsx`.

```
<!doctype html>
<html lang="fr">
 <head>
 <meta charset="UTF-8" />
 <link rel="icon" type="image/svg+xml" href="/vite.svg" />
 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
 <title>Vite + React</title>
 </head>
 <body>
 <!-- Root element where the React application will be rendered -->
```



```
<div id="root"></div>
<!-- Entry point script for the React application built with Vite -->
<script type="module" src="/src/main.jsx"></script>
</body>
</html>
```

## App.module.css

Found in folder: **/src**

This CSS file defines global and component-specific styles for a responsive web application layout. It sets a base style for all elements and styles for the header, title, and subtitle. It also creates a two-column grid layout for the main content that becomes a single column on smaller screens.

```
* {
 box-sizing: border-
box; /* Ensures padding and border are included in element's total width/height */
}

.app {
 min-height: 100vh; /* Makes the app container at least the full viewport height */
 background: #f1f5f9;
 font-family: -apple-system, BlinkMacSystemFont, 'Segoe UI', Roboto, sans-serif;
 color: #1e293b;
}

.header {
 padding: 28px 32px 20px;
 border-bottom: 1px solid #e2e8f0;
 background: #ffffff;
}

.title {
 font-size: 1.5rem;
 font-weight: 800;
 margin: 0 0 4px 0;
 color: #0f172a;
}

.subtitle {
 font-size: 0.9rem;
 color: #64748b;
 margin: 0;
}

.layout {
 display: grid;
 grid-template-columns: 1fr 380px; /* Main content area and a 380px sidebar */
 gap: 24px;
 padding: 28px 32px;
 max-width: 1200px; /* Constrains the layout width on large screens */
 margin: 0 auto; /* Centers the layout horizontally */
}
```

```
@media (max-width: 768px) {
 .layout {
 grid-template-columns: 1fr; /* Stacks columns vertically on mobile */
 padding: 16px;
 }
}
```

### CategorySummary.module.css

Found in folder: **/src/components**

This CSS code defines styles for a container and its components, including headings, lists, rows, and visual indicators like dots and warning badges. It uses flexbox for layout control and sets specific colors, spacing, and typography for each element. The styles are designed to create a clean, structured list with items that can include colored dots, labels, totals, and warning icons.

```
.container {
 margin-top: 20px; /* Adds spacing above the container */
}

.heading {
 font-size: 0.9rem;
 font-weight: 700;
 color: #64748b;
 text-transform: uppercase;
 letter-spacing: 0.05em;
 margin: 0 0 12px 0; /* Sets bottom margin for spacing below heading */
}

.list {
 list-style: none; /* Removes default list bullets */
 padding: 0;
 margin: 0;
 display: flex;
 flex-direction: column; /* Stacks list items vertically */
 gap: 8px; /* Adds consistent spacing between list items */
}

.row {
 display: flex;
 align-items: center; /* Vertically centers items within each row */
 gap: 8px; /* Spacing between row elements */
}

.dot {
 flex-shrink: 0; /* Prevents dot from shrinking in flex layout */
 width: 8px;
 height: 8px;
 border-radius: 50%; /* Makes the element a perfect circle */
}

.label {
```

```
flex: 1; /* Allows label to expand and fill available space */
font-size: 0.88rem;
color: #475569;
}

.total {
 font-size: 0.88rem;
 font-weight: 600;
 color: #dc2626; /* Red color for total values */
}

.warning {
 display: inline-flex;
 align-items: center;
 justify-content: center;
 flex-shrink: 0;
 width: 18px;
 height: 18px;
 border-radius: 50%; /* Circular warning badge */
 background: #dc2626; /* Red background */
 color: #ffffff;
 font-size: 0.75rem;
 font-weight: 700;
 line-height: 1; /* Ensures text fits neatly in circle */
}
```

## Dashboard.module.css

Found in folder: **/src/components**

This CSS styles a container for displaying financial statistics. It defines a clean, card-like layout with a heading and a grid of stat cards. The stat cards show income and expense values with distinct green and red colors.

```
.container {
 background: #ffffff;
 border-radius: 12px;
 padding: 24px;
 box-shadow: 0 1px 3px rgba(0, 0, 0, 0.08);
 height: 100%;
}

.heading {
 font-size: 1.1rem;
 font-weight: 700;
 color: #0f172a;
 margin: 0 0 16px 0;
 padding-bottom: 12px;
 border-bottom: 2px solid #f1f5f9;
}

.stats {
 display: grid;
```

```
 grid-template-columns: 1fr 1fr; /* Two-column grid for stat cards */
 gap: 12px;
 margin-bottom: 4px;
}

.statCard {
 background: #f8fafc;
 border-radius: 8px;
 padding: 14px;
}

.statLabel {
 font-size: 0.78rem;
 font-weight: 600;
 text-transform: uppercase;
 letter-spacing: 0.05em;
 color: #94a3b8;
 margin: 0 0 6px 0;
}

.statValue {
 font-size: 1.1rem;
 font-weight: 700;
 margin: 0;
}

.income {
 color: #16a34a; /* Green for positive/income values */
}

.expense {
 color: #dc2626; /* Red for negative/expense values */
}
```

## MonthlyBalance.module.css

Found in folder: **/src/components**

This CSS code defines styles for financial display components, including containers for profit and loss values. It sets up visual distinctions with different background colors, borders, and text colors for profit (green theme) versus loss (red theme) states. The styles also format labels, amounts, and status text with specific typography and spacing rules.

```
.container {
 border-radius: 10px;
 padding: 18px;
 text-align: center;
 margin-bottom: 20px;
}

/* Profit theme - green color scheme */
.profit {
 background: #f0fdf4;
```



```
border: 1px solid #bbf7d0;
}

/* Loss theme - red color scheme */
.loss {
 background: #fef2f2;
 border: 1px solid #fecaca;
}

/* Label styling for metric names */
.label {
 font-size: 0.82rem;
 font-weight: 600;
 text-transform: uppercase;
 letter-spacing: 0.05em;
 margin: 0 0 6px 0;
 color: #64748b;
}

/* Main amount display */
.amount {
 font-size: 1.6rem;
 font-weight: 800;
 margin: 0 0 4px 0;
 color: inherit;
}

/* Profit-specific amount color */
.profit .amount {
 color: #16a34a;
}

/* Loss-specific amount color */
.loss .amount {
 color: #dc2626;
}

/* Status text styling */
.status {
 font-size: 0.82rem;
 font-weight: 500;
 margin: 0;
}

/* Profit-specific status color */
.profit .status {
 color: #16a34a;
}

/* Loss-specific status color */
.loss .status {
 color: #dc2626;
}
```

## TransactionItem.module.css

Found in folder: **/src/components**

This CSS code styles list items to display financial transactions with colored dots, descriptions, categories, and amounts. It arranges these elements horizontally with spacing and borders between items, except the last one. Income and expense amounts are colored green and red respectively.

```
.item {
 display: flex;
 align-items: center;
 gap: 12px;
 padding: 12px 0;
 border-bottom: 1px solid #f1f5f9; /* Separator between items */
 list-style: none;
}

.item:last-child {
 border-bottom: none; /* Remove separator from last item */
}

.dot {
 flex-shrink: 0; /* Prevent dot from shrinking */
 width: 10px;
 height: 10px;
 border-radius: 50%; /* Circular dot */
}

.details {
 flex: 1; /* Take up remaining space */
 display: flex;
 flex-direction: column;
 gap: 2px;
}

.description {
 font-size: 0.95rem;
 font-weight: 500;
 color: #1e293b;
}

.category {
 font-size: 0.78rem;
 color: #94a3b8; /* Muted category text */
}

.amount {
 font-size: 0.95rem;
 font-weight: 600;
 white-space: nowrap; /* Prevent amount from wrapping */
}

.income {
 color: #16a34a; /* Green for income */
}
```

```
.expense {
 color: #dc2626; /* Red for expense */
}
```

### TransactionList.module.css

Found in folder: **/src/components**

This CSS code defines styles for a container, heading, search label, search input, and list components. It sets visual properties like colors, borders, spacing, and interactive states for a search interface. The styles create a clean, card-like layout with focused input feedback.

```
.container {
 background: #ffffff;
 border-radius: 12px;
 padding: 24px;
 box-shadow: 0 1px 3px rgba(0, 0, 0, 0.08);
 height: 100%;
}

.heading {
 font-size: 1.1rem;
 font-weight: 700;
 color: #0f172a;
 margin: 0 0 14px 0;
 padding-bottom: 12px;
 border-bottom: 2px solid #f1f5f9; /* Subtle separator below heading */
}

.searchLabel {
 display: block;
 margin-bottom: 6px;
 font-size: 0.82rem;
 font-weight: 600;
 color: #475569;
}

.searchInput {
 width: 100%;
 margin-bottom: 16px;
 border: 1px solid #cbd5e1;
 border-radius: 8px;
 padding: 10px 12px;
 font: inherit;
 color: #0f172a;
 background: #ffffff;
}

.searchInput::placeholder {
 color: #94a3b8; /* Placeholder text color */
}
```



```
.searchInput:focus {
 border-color: #64748b;
 outline: 3px solid #e2e8f0; /* Custom focus outline */
}

.list {
 padding: 0;
 margin: 0;
}
```

## monthlyBudgets.js

Found in folder: **/src/data**

This code defines an object containing budget categories and their monthly limits, then exports it as the default export for use in other modules.

```
const monthlyBudgets = {
 Housing: 900, // Rent or mortgage
 Food: 200, // Groceries and dining
 Entertainment: 50, // Movies, subscriptions
 Utilities: 100, // Electricity, water, internet
 Health: 60, // Medications, insurance
 Shopping: 75, // Clothing, miscellaneous
 Transport: 75, // Fuel, public transit
};

export default monthlyBudgets;
```

## transactions.js

Found in folder: **/src/data**

This code defines an array of transaction objects with properties like description, amount, category, and month, then exports it as the default export. Each transaction represents either income (positive amount) or expense (negative amount) for the month of January. The exported array can be imported and used elsewhere in the project.

```
// Array of transaction objects for January
const transactions = [
 { id: 1, description: 'Salary', amount: 3200, category: 'Income', month: 'January' },
 { id: 2, description: 'Rent', amount: -950, category: 'Housing', month: 'January' },
 { id: 3, description: 'Groceries', amount: -120, category: 'Food', month: 'January' },
 { id: 4, description: 'Netflix', amount: -15, category: 'Entertainment', month: 'January' },
 { id: 5, description: 'Electricity bill', amount: -85, category: 'Utilities', month: 'January' },
 { id: 6, description: 'Freelance project', amount: 800, category: 'Income', month: 'January' }
];

export default transactions;
```



```
'January' },
 { id: 7, description: 'Restaurant dinner', amount: -65, category: 'Food', month: 'January' },
 { id: 8, description: 'Gym membership', amount: -40, category: 'Health', month: 'January' },
 { id: 9, description: 'Internet bill', amount: -35, category: 'Utilities', month: 'January' },
 { id: 10, description: 'Coffee shops', amount: -45, category: 'Food', month: 'January' },
 { id: 11, description: 'Amazon purchase', amount: -78, category: 'Shopping', month: 'January' },
 { id: 12, description: 'Bus pass', amount: -60, category: 'Transport', month: 'January' },
];

// Export as default for use in other modules
export default transactions;
```

### **transactionService.js**

Found in folder: **/src/services**

This code provides functions to analyze financial transaction data. It calculates totals, summaries, and alerts based on transaction categories and monthly budgets. The functions help track income, expenses, balances, and budget compliance.

```
import transactions from '../data/transactions.js';
import monthlyBudgets from '../data/monthlyBudgets.js';

export function getAllTransactions() {
 return transactions;
}

export function getTransactionsByCategory(category) {
 return transactions.filter((t) => t.category === category);
}

export function getTotalIncome() {
 return transactions
 .filter((t) => t.amount > 0) // Positive amounts represent income
 .reduce((sum, t) => sum + t.amount, 0);
}

export function getTotalExpenses() {
 return transactions
 .filter((t) => t.amount < 0) // Negative amounts represent expenses
 .reduce((sum, t) => sum + t.amount, 0);
}

export function getBalance() {
 return getTotalIncome() + getTotalExpenses(); // Net balance calculation
}
```

```
export function getSummaryByCategory() {
 const map = {};
 for (const t of transactions) {
 if (!map[t.category]) {
 map[t.category] = { category: t.category, total: 0 };
 }
 map[t.category].total += t.amount;
 }
 return Object.values(map);
}

export function getExpenseSummaryByCategory() {
 return getSummaryByCategory()
 .filter((item) => item.total < 0) // Only include expense categories
 .map((item) => {
 const spent = Math.abs(item.total);
 const budgetLimit = monthlyBudgets[item.category] ?? null;
 const isOverBudget = budgetLimit !== null && spent > budgetLimit;

 return {
 ...item,
 spent,
 budgetLimit,
 isOverBudget,
 overBudgetBy: isOverBudget ? spent - budgetLimit : 0,
 };
 });
}

export function getBudgetAlertsByCategory() {
 return getExpenseSummaryByCategory().filter((item) => item.isOverBudget);
}

export function getTopSpendingCategory() {
 const [topCategory] = getExpenseSummaryByCategory().sort((a, b) => b.spent - a.spent);

 return topCategory.category;
}

export function getLowestSpendingCategory() {
 const [lowestCategory] = getExpenseSummaryByCategory().sort((a, b) => a.spent - b.spent);

 return lowestCategory.category;
}

export function getAverageDailySpend() {
 return Number((Math.abs(getTotalExpenses()) / 31).toFixed(2)); // Assumes 31-day month
}
```

**transactionService.test.js**

Found in folder: **/src/services**

This code tests various financial transaction service functions, verifying they return correct data like totals, category summaries, and budget alerts. It checks that functions handle edge cases, such as unknown categories, and validate specific expected values like the top spending category and average daily spend.

```
import { describe, it, expect } from 'vitest';
import {
 getAllTransactions,
 getTotalIncome,
 getTotalExpenses,
 getBalance,
 getTransactionsByCategory,
 getSummaryByCategory,
 getExpenseSummaryByCategory,
 getBudgetAlertsByCategory,
 getTopSpendingCategory,
 getAverageDailySpend,
} from '../transactionService.js';

describe('transactionService', () => {
 it('getAllTransactions returns all 12 transactions', () => {
 expect(getAllTransactions()).toHaveLength(12);
 });

 it('getTotalIncome returns the correct sum', () => {
 expect(getTotalIncome()).toBe(4000);
 });

 it('getTotalExpenses returns the correct sum', () => {
 expect(getTotalExpenses()).toBe(-1493);
 });

 it('getBalance returns the correct balance', () => {
 expect(getBalance()).toBe(2507);
 });

 it('getTransactionsByCategory returns only matching transactions', () => {
 const food = getTransactionsByCategory('Food');
 expect(food).toHaveLength(3);
 expect(food.every((t) => t.category === 'Food')).toBe(true);
 });

 it('getTransactionsByCategory returns empty array for unknown category', () => {
 expect(getTransactionsByCategory('Unknown')).toEqual([]);
 });

 it('getSummaryByCategory returns correct number of categories', () => {
 expect(getSummaryByCategory()).toHaveLength(8);
 });

 it('getSummaryByCategory returns correct total for Food category', () => {
 const summary = getSummaryByCategory();
 const food = summary.find((item) => item.category === 'Food');
```



```
 expect(food.total).toBe(-230);
 });

 it('getExpenseSummaryByCategory returns only expense categories with budget status'
, () => {
 const summary = getExpenseSummaryByCategory();

 expect(summary).toHaveLength(7);
 // Ensure all items are expenses (negative totals)
 expect(summary.every((item) => item.total < 0)).toBe(true);
 // Check that budget status is included
 expect(summary.every((item) => typeof item.isOverBudget === 'boolean')).toBe(true
);
 });

 it('getBudgetAlertsByCategory returns categories over budget', () => {
 expect(getBudgetAlertsByCategory()).toEqual([
 {
 category: 'Housing',
 total: -950,
 spent: 950,
 budgetLimit: 900,
 isOverBudget: true,
 overBudgetBy: 50,
 },
 {
 category: 'Food',
 total: -230,
 spent: 230,
 budgetLimit: 200,
 isOverBudget: true,
 overBudgetBy: 30,
 },
 {
 category: 'Utilities',
 total: -120,
 spent: 120,
 budgetLimit: 100,
 isOverBudget: true,
 overBudgetBy: 20,
 },
 {
 category: 'Shopping',
 total: -78,
 spent: 78,
 budgetLimit: 75,
 isOverBudget: true,
 overBudgetBy: 3,
 },
]
);
 });

 it('getTopSpendingCategory returns the category with the highest total expenses', (
) => {
 expect(getTopSpendingCategory()).toBe('Housing');
```

```
});

it('getAverageDailySpend returns the average expense spend over 31 days', () => {
 expect(getAverageDailySpend()).toBe(48.16);
});
});
```

### **categoryUtils.js**

Found in folder: **/src/utils**

This code defines a mapping of category names to specific hex colour codes and provides a function to retrieve the colour for a given category, defaulting to a fallback colour if the category is not found.

```
const CATEGORY_COLOURS = {
 Income: '#22c55e',
 Housing: '#3b82f6',
 Food: '#f97316',
 Entertainment: '#a855f7',
 Utilities: '#eab308',
 Health: '#ef4444',
 Shopping: '#06b6d4',
 Transport: '#14b8a6',
};

const FALLBACK_COLOUR = '#6b7280'; // Default colour for unknown categories

export function getCategoryColour(category) {
 // Return the mapped colour or fallback if category not found
 return CATEGORY_COLOURS[category] ?? FALLBACK_COLOUR;
}
```

### **categoryUtils.test.js**

Found in folder: **/src/utils**

This code tests a function that returns color strings for categories. It verifies that known categories receive a string color and that unknown categories get a specific fallback color.

```
import { describe, it, expect } from 'vitest';
import { getCategoryColour } from '../categoryUtils.js';

// Predefined list of categories expected to have valid colors
const KNOWN_CATEGORIES = [
 'Income',
 'Housing',
 'Food',
 'Entertainment',
 'Utilities',
 'Health',
 'Shopping',
];
```

```
 'Transport',
];

describe('categoryUtils', () => {
 // Ensures the function returns a string for each category in the known list
 it('getCategoryColour returns a string for every known category', () => {
 for (const category of KNOWN_CATEGORIES) {
 expect(typeof getCategoryColour(category)).toBe('string');
 }
 });

 // Tests that an unrecognized category receives the designated fallback color
 it('getCategoryColour returns a fallback value for an unknown category', () => {
 expect(getCategoryColour('Unknown')).toBe('#6b7280');
 });
});
```

### currencyUtils.js

Found in folder: **/src/utils**

This function formats a numeric amount as a British Pound currency string. It ensures the amount is displayed with two decimal places and includes thousand separators. A negative sign is prepended for negative amounts, while the pound symbol is always placed at the front.

```
export function formatCurrency(amount) {
 // Get absolute value to format digits independently of sign
 const abs = Math.abs(amount);
 // Format to 2 decimal places and insert thousand separators
 const formatted = abs.toFixed(2).replace(/\B(?=(\d{3})+(?!\d))/g, ',');
 // Add currency symbol and restore negative sign if needed
 return amount < 0 ? `-${formatted}` : `£${formatted}`;
}
```

### currencyUtils.test.js

Found in folder: **/src/utils**

This code tests the `formatCurrency`` function to ensure it correctly formats positive and negative numbers into UK currency strings with pound symbols, commas for thousands, and two decimal places.

```
import { describe, it, expect } from 'vitest';
import { formatCurrency } from './currencyUtils.js';

// Test suite for currency formatting utilities
describe('currencyUtils', () => {
 // Tests formatting of positive numbers
 it('formatCurrency formats a positive number correctly', () => {
 expect(formatCurrency(3200)).toBe('£3,200.00');
 });
});
```

```
// Tests formatting of negative numbers
it('formatCurrency formats a negative number correctly', () => {
 expect(formatCurrency(-950)).toBe('-£950.00');
});
});
```

## Weather Dashboard

### AGENTS.md

Found in folder: /

This file provides project documentation for a weather dashboard, outlining its purpose, technology stack, development commands, and coding conventions. It specifies the project's structure and critical rules that must be followed during development. The document serves as a reference for students working on the codebase.

# Weather Dashboard – AGENTS.md

#### ## Project

A React weather dashboard built as a starter project for a course on AI-assisted development. Students use this codebase to practice finding and fixing bugs with AI tooling.

#### ## Stack

- React 19 + Vite 7
- CSS Modules – no external UI libraries
- Vitest for unit testing

#### ## Commands

Command	Description
`npm run dev`	Start development server
`npm run build`	Production build
`npm run test`	Run Vitest test suite

#### ## Conventions

- All components live in `src/components/` with a co-located CSS module
- Shared utilities live in `src/utils/`
- Mock weather data lives in `src/data/`
- CSS class names use camelCase inside module files
- Component files use PascalCase; utility and data files use camelCase

#### ## Critical Rules

- Do not add external dependencies beyond React, Vite, and Vitest
- Do not add Tailwind or any CSS framework or component library
- Do not add routing or localStorage



- Do not replace the mock data with real API calls
- The broken behaviours in this repo are intentional student exercises – do not silently fix them
- Before the final commit of any session, run `/review` and then `git diff` to summarise every change made in plain English.

## README.md

Found in folder: /

This is a README file that provides an overview of the Weather Dashboard project. It describes the application's purpose, lists supported cities, and gives setup instructions. It also outlines the project's technology stack and testing commands.

### # Weather Dashboard

A simple weather dashboard built with React and Vite. Type a city name into the search bar to view current weather conditions including temperature in both Celsius and Fahrenheit, wind speed, and sky condition.

### ## Supported Cities

London, New York, Tokyo, Paris, Sydney

Searches are case-insensitive. Entering an unrecognised city shows a helpful error message.

### ## Getting Started

```
```bash
npm install
npm run dev
```
```

Open [<http://localhost:5173>](<http://localhost:5173>) in your browser.

### ## Running Tests

```
```bash
npm run test
```
```

### ## Tech Stack

- [React](<https://react.dev/>) – UI library
- [Vite](<https://vitejs.dev/>) – build tool and dev server
- [Vitest](<https://vitest.dev/>) – unit testing framework
- Plain CSS Modules – no external UI libraries

## SKILL.md

Found in folder: **/.codex/skills/debug-and-verify**

This code defines a structured workflow for debugging failing tests in a local project, guiding the user from analyzing test output to making incremental fixes and committing verified changes. It provides step-by-step instructions to identify failures, inspect code, apply focused fixes, and verify results through repeated test runs. The workflow concludes with steps to review changes and commit fixes only after all tests pass.

```

name: debug-and-verify
description: Use when given raw failing test output for a local project and asked to
debug it. Fix one failure at a time, run npm run test after each fix, review the diff
, and commit only after confirming all tests pass.

```

# Debug And Verify

## Workflow

Use this skill when the user provides raw `npm run test` output or asks to fix failing tests in this project.

1. Read the failure output carefully.
  - Identify the failing file, test name, assertion or parser error, expected value, received value, and stack location.
  - Start with the first concrete failure. Do not guess at unrelated bugs.
2. Inspect the smallest relevant code surface.
  - Read the failing test and the implementation it exercises.
  - Use `rg` first for related symbols, messages, and imports.
  - Respect project instructions and avoid silently fixing unrelated intentional exercises.
3. Make one focused fix.
  - Keep edits limited to the files needed for the current failure.
  - Preserve existing style and project conventions.
  - Do not add dependencies, routing, localStorage, real APIs, UI libraries, or CSS frameworks.
4. Verify incrementally.
  - Run `npm run test` immediately after each fix.
  - If tests still fail, treat the new output as the next input and repeat from step 1.
  - Continue until `npm run test` exits successfully.
5. Review the diff before committing.
  - Run a diff command such as `git diff -- <changed-files>` when the project is a git repo.
  - Confirm the diff only contains the intended fixes.
  - If unrelated user changes are present, leave them untouched and do not include them in the commit.
6. Commit the verified fix.
  - Check git status.
  - Stage only the files changed for this debugging task.

- Commit with a message that confirms tests pass, for example:

```
```bash
git commit -m "Fix failing tests; npm run test passes"
```
```

## ## Reporting

In the final response, include:

- The files changed.
- The final `npm run test` result.
- The commit hash, if a commit was created.
- Any reason a commit could not be created, such as the project not being a git repo.

## App.jsx

Found in folder: **/src**

This code provides a user interface for searching and displaying weather data. It manages loading states and errors while fetching weather information for a given city. The search functionality includes a debouncing mechanism to prevent outdated results from being displayed.

```
import { useRef, useState } from 'react'
import SearchBar from './components/SearchBar'
import WeatherCard from './components/WeatherCard'
import ErrorMessage from './components/ErrorMessage'
import LoadingSpinner from './components/LoadingSpinner'
import { getWeatherByCity } from './data/weatherData'
import styles from './App.module.css'

function App() {
 const [weatherData, setWeatherData] = useState(null)
 const [error, setError] = useState(false)
 const [isLoading, setIsLoading] = useState(false)
 // Tracks the most recent search to ignore outdated responses
 const latestSearchId = useRef(0)

 const handleSearch = (city) => {
 // Increment search ID to identify the current request
 const searchId = latestSearchId.current + 1
 latestSearchId.current = searchId

 setIsLoading(true)
 setWeatherData(null)
 setError(false)

 // Simulate network delay and implement debouncing
 setTimeout(() => {
 // Ignore this response if a newer search has started
 if (searchId !== latestSearchId.current) {
 return
 }
 }, 1000)

 getWeatherByCity(city).then((data) => {
 setWeatherData(data)
 setError(false)
 }).catch((error) => {
 setError(true)
 })
 }

 return (
 <div>
 <SearchBar onSearch={handleSearch} />
 <div>
 <WeatherCard data={weatherData} />
 <ErrorMessage error={error} />
 </div>
 <LoadingSpinner loading={isLoading} />
 </div>
)
}
```



```
 const data = getWeatherByCity(city)
 if (data) {
 setWeatherData(data)
 setError(false)
 } else {
 setWeatherData(null)
 setError(true)
 }
 setIsLoading(false)
 }, 600)
}

return (
 <div className={styles.container}>
 <h1 className={styles.title}>Weather Dashboard</h1>
 <SearchBar onSearch={handleSearch} />
 {isLoading && <LoadingSpinner />}
 {!isLoading && error && <ErrorMessage />}
 {!isLoading && weatherData && <WeatherCard data={weatherData} />}
 </div>
)
}

export default App
```

## main.jsx

Found in folder: **/src**

This code initializes a React application by rendering the main App component into the DOM. It uses React's StrictMode to highlight potential problems during development. The App component is mounted inside the HTML element with the id 'root'.

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import App from './App.jsx';

// Create a React root for the DOM element with id 'root'
const root = createRoot(document.getElementById('root'));

// Render the App component inside StrictMode for development checks
root.render(
 <StrictMode>
 <App />
 </StrictMode>
);
```

## ErrorMessage.jsx

Found in folder: **/src/components**

This component renders a styled error message when a city search fails, suggesting example cities to try. It uses CSS Modules for styling by applying a class from the imported stylesheet. The component is then exported for use elsewhere in the application.

```
import styles from './ErrorMessage.module.css'

function ErrorMessage() {
 return (
 // Displays a user-friendly error message with suggested city names
 <div className={styles.error}>
 City not found. Try London, New York, Tokyo, Paris or Sydney.
 </div>
)
}

export default ErrorMessage
```

## LoadingSpinner.jsx

Found in folder: **/src/components**

This code defines a React component that displays a loading spinner with an accessible announcement while weather data is being fetched. It uses CSS modules for styling and includes ARIA attributes to properly communicate its state to screen readers. The component exports a single functional element that visually and accessibly indicates a loading process.

```
import styles from './LoadingSpinner.module.css'

function LoadingSpinner() {
 return (
 // Container with accessibility attributes to announce loading state
 <div className={styles.loading} role="status" aria-live="polite">
 {/* Visual spinner element hidden from screen readers */}

 Loading weather data...
 </div>
)
}

export default LoadingSpinner
```

## SearchBar.jsx

Found in folder: **/src/components**

This code defines a React component that renders a search form with an input field and a submit button. It uses local state to track the input value and triggers a provided `onSearch` function when the form is submitted with a non-empty, trimmed value. The component prevents the default form submission behavior and clears any extra whitespace from the input before passing it to the search handler.



```
import { useState } from 'react'
import styles from './SearchBar.module.css'

// SearchBar component that takes an onSearch callback prop
function SearchBar({ onSearch }) {
 // State to manage the value of the search input field
 const [inputValue, setInputValue] = useState('')

 // Handles form submission: prevents default, validates input, and calls onSearch
 const handleSubmit = (e) => {
 e.preventDefault()
 // Only search if the trimmed input is not empty
 if (inputValue.trim()) {
 onSearch(inputValue.trim())
 }
 }

 return (
 // Form that triggers handleSubmit on submission
 <form className={styles.form} onSubmit={handleSubmit}>
 <input
 className={styles.input}
 type="text"
 placeholder="Enter city name..."
 value={inputValue}
 onChange={(e) => setInputValue(e.target.value)}
 />
 <button className={styles.button} type="submit">
 Search
 </button>
 </form>
)
}

export default SearchBar
```

## WeatherCard.jsx

Found in folder: **/src/components**

This code renders a weather card component displaying city, condition, temperature in Celsius and Fahrenheit, and wind speed. It converts the temperature from Celsius to Fahrenheit using a utility function. The component receives weather data as a prop and applies CSS module styles for presentation.

```
import { celsiusToFahrenheit } from '../utils/temperatureUtils'
import styles from './WeatherCard.module.css'

function WeatherCard({ data }) {
 // Convert temperature to Fahrenheit and round to nearest integer
 const fahrenheit = Math.round(celsiusToFahrenheit(data.temperature))

 return (
```



```
<div className={styles.card}>
 <h2 className={styles.city}>{data.city}</h2>
 <p className={styles.condition}>{data.condition}</p>
 <div className={styles.temperature}>
 {data.temperature}°C
 {fahrenheit}°F
 </div>
 <div className={styles.wind}>
 Wind Speed
 {data.windSpeed} km/h
 </div>
</div>
)
}

export default WeatherCard
```

## index.html

Found in folder: /

This HTML file sets up the basic structure of a web page, including metadata and viewport settings for responsive design. It links a favicon and loads a React application by mounting it into a `div` element with the id "root". The page's JavaScript entry point is specified as a module script.

```
<!doctype html>
<html lang="fr">
 <head>
 <meta charset="UTF-8" />
 <link rel="icon" type="image/svg+xml" href="/vite.svg" />
 <meta name="viewport" content="width=device-width, initial-scale=1.0" />
 <title>Vite + React</title>
 </head>
 <body>
 <!-- Root container where the React app will be rendered -->
 <div id="root"></div>
 <!-- Entry point for the React application built with Vite -->
 <script type="module" src="/src/main.jsx"></script>
 </body>
</html>
```

## App.module.css

Found in folder: /src

This CSS defines two classes: `.container` creates a full-height, centered column layout with padding and a light background, while `.title` styles a large, bold, dark heading with bottom margin.

```
.container {
 min-height: 100vh; /* Ensures container fills at least the full viewport height */
 display: flex;
```

```
flex-direction: column;
align-items: center;
padding: 40px 20px;
background-color: #f0f4f8;
font-family: 'Segoe UI', system-ui, sans-serif;
box-sizing: border-box; /* Padding included in width/height calculation */
}

.title {
 font-size: 2rem;
 color: #1a202c;
 margin: 0 0 32px; /* Top, horizontal, bottom margin */
 font-weight: 700;
}
```

### **ErrorMessage.module.css**

Found in folder: **/src/components**

This CSS defines a visual style for error messages, making them stand out with a red color scheme and rounded borders. It ensures the error container is responsive but doesn't exceed a maximum width. The styling includes padding, a light red background, and a darker red border for clear visual feedback.

```
.error {
 width: 100%; /* Takes full width of its container */
 max-width: 480px; /* But doesn't exceed this maximum width */
 padding: 16px;
 background-color: #fff5f5; /* Light red background */
 border: 1px solid #feb2b2; /* Medium red border */
 border-radius: 8px; /* Rounded corners */
 color: #c53030; /* Dark red text color */
 font-size: 0.9rem;
 text-align: center;
}
```

### **LoadingSpinner.module.css**

Found in folder: **/src/components**

This CSS defines a loading container with a centered spinner animation. The `.loading` class creates a styled box with flex alignment and shadow, while .spinner` creates an animated circular element using a rotating border. The @keyframes spin` rule continuously rotates the spinner to indicate loading activity.`

```
.loading {
 width: 100%;
 max-width: 400px;
 display: flex;
 align-items: center;
 justify-content: center;
```



```
gap: 12px; /* Space between spinner and text */
padding: 24px;
background: white;
border-radius: 16px;
box-shadow: 0 4px 24px rgba(0, 0, 0, 0.1);
color: #2d3748;
font-size: 0.95rem;
font-weight: 600;
box-sizing: border-box; /* Include padding in width calculation */
}

.spinner {
 width: 24px;
 height: 24px;
 border: 3px solid #bee3f8; /* Light blue background ring */
 border-top-color: #4299e1; /* Dark blue animated segment */
 border-radius: 50%;
 animation: spin 0.8s linear infinite; /* Continuous rotation */
 flex: 0 0 auto; /* Prevent spinner from stretching */
}

@keyframes spin {
 to {
 transform: rotate(360deg); /* Full rotation */
 }
}
```

## SearchBar.module.css

Found in folder: **/src/components**

This CSS code styles a form container, an input field, and a button. It defines their layout, visual appearance, and interactive states like focus and hover. The styles create a responsive, modern UI with smooth transitions.

```
.form {
 display: flex;
 gap: 8px; /* Spacing between child elements */
 width: 100%;
 max-width: 480px; /* Constrains form width on larger screens */
 margin-bottom: 24px; /* Space below the form */
}

.input {
 flex: 1; /* Allows input to grow and fill available space */
 padding: 12px 16px;
 font-size: 1rem;
 border: 2px solid #cbd5e0;
 border-radius: 8px;
 outline: none;
 transition: border-color 0.2s; /* Smooth color change on focus */
 font-family: inherit;
}
```

```
.input:focus {
 border-color: #4299e1; /* Highlight color when input is focused */
}

.button {
 padding: 12px 24px;
 font-size: 1rem;
 font-weight: 600;
 color: white;
 background-color: #4299e1;
 border: none;
 border-radius: 8px;
 cursor: pointer;
 transition: background-color 0.2s; /* Smooth color change on hover */
 font-family: inherit;
}

.button:hover {
 background-color: #3182ce; /* Darker shade on hover */
}
```

## WeatherCard.module.css

Found in folder: **/src/components**

This CSS styles a weather display card with a clean, centered layout. It defines visual styles for the city name, condition, temperature in Celsius and Fahrenheit, and a wind information section. The card has a white background, rounded corners, and a subtle shadow.

```
.card {
 width: 100%;
 max-width: 400px; /* Limits card width on larger screens */
 background: white;
 border-radius: 16px; /* Creates rounded corners */
 padding: 32px;
 box-shadow: 0 4px 24px rgba(0, 0, 0, 0.1); /* Adds a soft shadow */
 text-align: center;
}

.city {
 font-size: 1.75rem;
 font-weight: 700;
 color: #1a202c;
 margin: 0 0 8px; /* Bottom margin only */
}

.condition {
 font-size: 1rem;
 color: #718096;
 margin: 0 0 24px; /* Bottom margin only */
}
```

```
.temperature {
 display: flex;
 justify-content: center;
 gap: 24px; /* Space between Celsius and Fahrenheit */
 margin-bottom: 24px;
}

.tempC {
 font-size: 2.5rem;
 font-weight: 700;
 color: #2d3748;
}

.tempF {
 font-size: 2.5rem;
 font-weight: 300; /* Lighter weight to differentiate from Celsius */
 color: #a0aec0;
}

.wind {
 display: flex;
 justify-content: space-between; /* Separates label and value */
 padding: 16px;
 background: #f7fafc; /* Light background for the wind section */
 border-radius: 8px;
}

.windLabel {
 color: #718096;
 font-size: 0.875rem;
}

.windValue {
 color: #2d3748;
 font-weight: 600;
 font-size: 0.875rem;
}
```

## **weatherData.js**

Found in folder: **/src/data**

This code defines a set of hardcoded weather data for five cities and exports a function to retrieve that data. The ``getWeatherByCity`` function takes a city name as input, converts it to lowercase, and returns the corresponding weather object. If the city is not found in the data, it returns ``null``.

```
const weatherData = {
 london: {
 city: 'London',
 temperature: 18,
 windSpeed: 12,
 condition: 'Partly Cloudy',
 },
}
```



```
'new york': {
 city: 'New York',
 temperature: 24,
 windSpeed: 8,
 condition: 'Sunny',
},
tokyo: {
 city: 'Tokyo',
 temperature: 21,
 windSpeed: 15,
 condition: 'Overcast',
},
paris: {
 city: 'Paris',
 temperature: 16,
 windSpeed: 10,
 condition: 'Light Rain',
},
sydney: {
 city: 'Sydney',
 temperature: 28,
 windSpeed: 20,
 condition: 'Clear Sky',
},
}

// Returns weather data for a given city name, or null if not found
export const getWeatherByCity = (cityName) => {
 // Lookup is case-insensitive; convert input to lowercase to match object keys
 return weatherData[cityName.toLowerCase()] ?? null
}
```

### **weatherData.test.js**

Found in folder: **/src/data**

This code tests the `getWeatherByCity` function using the Vitest framework. It verifies that the function returns correct weather data for a known valid city. It also checks that the function returns null when given an invalid city name.

```
import { describe, expect, it } from 'vitest'
import { getWeatherByCity } from '../weatherData'

// Test suite for the getWeatherByCity function
describe('getWeatherByCity', () => {
 // Tests the function with a valid city input
 it('returns weather data for a valid city', () => {
 expect(getWeatherByCity('London')).toEqual({
 city: 'London',
 temperature: 18,
 windSpeed: 12,
 condition: 'Partly Cloudy',
 })
 })
})
```

```
 })

 // Tests the function with an invalid city input
 it('returns null for an invalid city', () => {
 expect(getWeatherByCity('Atlantis')).toBeNull()
 })
})
```

### **temperatureUtils.js**

Found in folder: **/src/utils**

This function converts a temperature from Celsius to Fahrenheit. It takes a Celsius value as input and returns the equivalent Fahrenheit value using the standard conversion formula.

```
export const celsiusToFahrenheit = (celsius) => {
 // Convert Celsius to Fahrenheit using the formula: (C × 9/5) + 32
 return celsius * 1.8 + 32
}
```

### **temperatureUtils.test.js**

Found in folder: **/src/utils**

This code tests a temperature conversion function by verifying that 100 degrees Celsius correctly converts to 212 degrees Fahrenheit. It uses a testing framework to define a test case and an assertion to check the result.

```
import { describe, it, expect } from 'vitest'
import { celsiusToFahrenheit } from './temperatureUtils'

// Test suite for the celsiusToFahrenheit conversion function
describe('celsiusToFahrenheit', () => {
 // Tests a specific conversion: 100°C should equal 212°F
 it('converts 100°C to 212°F', () => {
 expect(celsiusToFahrenheit(100)).toBe(212)
 })
})
```